

Advanced Programming

Introduction to Java language



Java language

- The 1st version was made available in 1995 by *Sun Microsystems*
 - Its study started in 1991
- Fully object-oriented language
- Cross-platform
- The syntax follows the C/C++ style



Java language

- Objectives

(<https://www.oracle.com/java/technologies/introduction-to-java.html>)

- Simple, Object Oriented and Familiar
- Robust and Secure
- Architecture Neutral and Portable
- High Performance
- Interpreted, Threaded (multitasking) and Dynamic

Java Development Kit (JDK)

- All the essential tools required to compile, debug, execute, and deploy Java applications are provided by the:

Java Development Kit (JDK)



JDK installation

- *OracleJDK vs OpenJDK*



- *OracleJDK*

- Commercial version, requiring licensing
 - Support provided by *Oracle*
 - Supposedly more stable and with a better performance

- *OpenJDK*

- *Open source* (we can contribute to its development)
 - Although its support is primarily done by its active development communities, there are specific distributions with their own support

JDK installation

- JDK version to install: Java SE
 - The latest available version is 23
 - *"JDK 23 will receive updates under these terms, until March 2025, when it will be superseded by JDK 24."*
- Available at:
 - OracleJDK
 - <https://www.oracle.com/java/technologies/downloads/>
 - OpenJDK
 - <https://jdk.java.net/23/>
- Version 21 is a *Long-Term Support* (LTS) release and will be supported until September 2026

Compiling a Java program

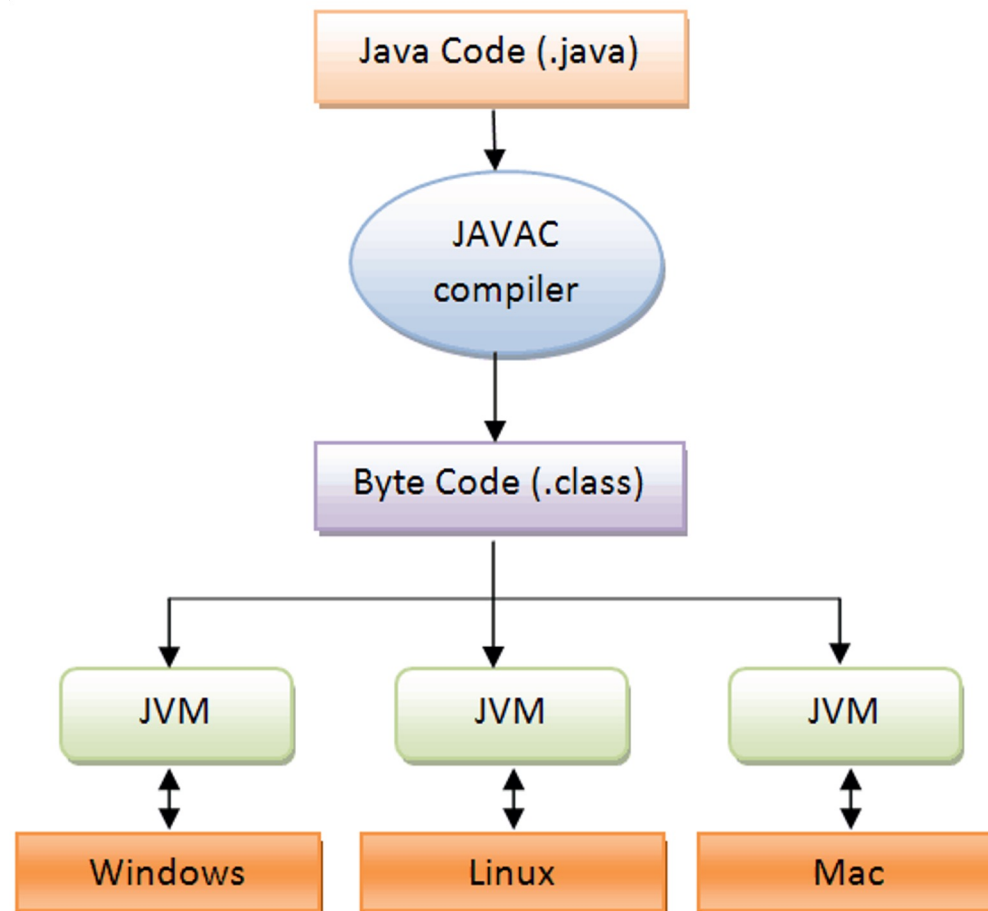
- Compilation, which includes syntax checking and generation of compiled code, is performed using the JDK
 - Some development environments have built-in versions
 - Includes tools and class libraries
 - Includes a compilation tool: `javac`
 - Intermediate code, called *Java bytecode*, is generated
 - Architecture independent code

Java Virtual Machine

- As the compilation result is not native code, it is necessary to have a *Java bytecode interpreter*
- The *Java bytecode interpreter* is called *Java Virtual Machine* (JVM)
 - The JDK includes the "java" command to run programs previously compiled to *Java bytecode* (.class files)
 - The JVM provides a secure, isolated environment in which programs run without affecting or being affected by other programs (*sandbox*)

Java Virtual Machine

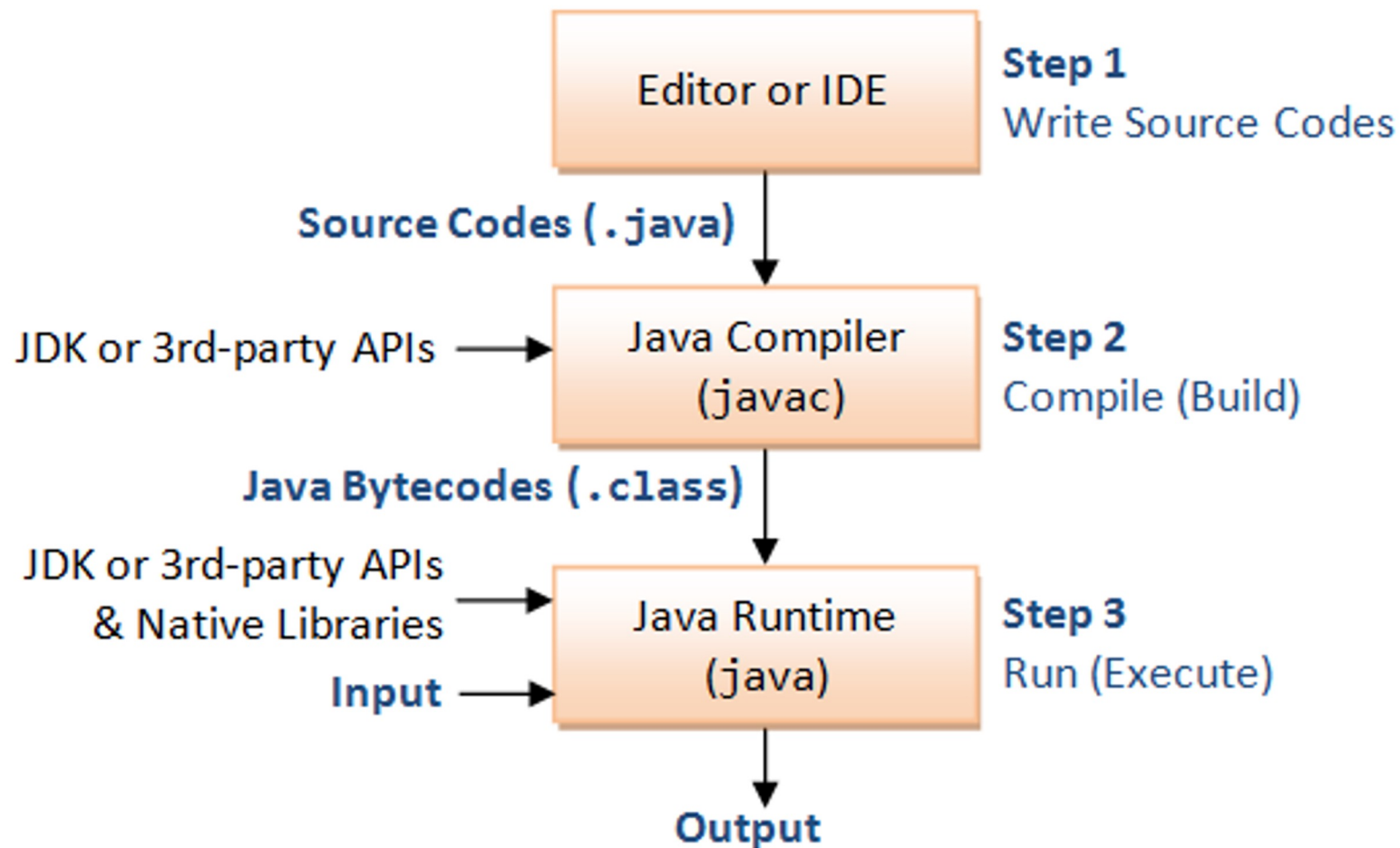
- For Java programs to run on a particular platform, a JVM for that platform must exist



Java Runtime Environment (JRE)

- Note that it is not necessary to install the JDK on machines where you just want to run the programs, just install a simplified version that only supports execution, called *Java Runtime Environment* (JRE)

Creation and execution



Example

```
// Example of a Java program
// Filename: Example1.java

public class Example1 {
    public static void main(String args[]) {
        System.out.println("Java@DEIS-ISEC");
    }
}
```


About the example

- All code is encapsulated in classes or similar
- A public class (in the example: `class Example1`) is defined in a file with the same name as that class and the extension `.java`
- There can be several classes in the same file, but only one can be public
- The first function to be executed in a *Java program* is `public static void main(String args[])`

Example (compilation and execution)

- Compilation is performed by doing
 - `javac Example1.java`
 - In case of success, the file `Example1.class` is created
 - If there are errors, these are indicated through the line number in the file in question.
- The execution is carried out by doing:
 - `java Example1`

Example (compilation and execution)

```
PA@deis$ cat Example1.java
```

```
public class Example1 {  
    public static void main(String args[]) {  
        System.out.println("Java@DEIS-ISEC");  
    }  
}
```

```
PA@deis$ javac Example1.java
```

```
PA@deis$ java Example1
```

```
Java@DEIS-ISEC
```

```
PA@deis$
```

Java bytecode

- Use the `javap` command to display the *Java bytecode*

```
PA@deis$ javap -c Example1
```

```
Compiled from "Example1.java"
```

```
public class Example1 {
```

```
    public Example1();
```

```
        Code:
```

```
            0: aload_0
```

```
            1: invokespecial #1          // Method java/lang/Object."<init>":()V
```

```
            4: return
```

```
public static void main(java.lang.String[]);
```

```
    Code:
```

```
        0: getstatic      #7          // Field java/lang/System.out:Ljava/io/PrintStream;
```

```
        3: ldc           #13         // String Java@DEIS-ISEC
```

```
        5: invokevirtual #15         // Method java/io/PrintStream.println:(Ljava/lang/String;)V
```

```
        8: return
```

```
}
```

```
PA@deis$
```

Java Language – Basic Concepts

Variables and primitive types

Arrays

Operators

Flow control

<https://dev.java/learn/language-basics>

Primitive data types

- Although the *Java* language is fully object-oriented, the following "primitive data types" are supported:
 - `byte` (*8 bits*): -128 ... 127
 - `short` (*16 bits*): -32768 ... 32767
 - `int` (*32 bits*): $-2^{31} \dots 2^{31} - 1$
 - `long` (*64 bits*): $-2^{63} \dots 2^{63} - 1$
 - `float` (*IEEE 754 32 bits*)
 - `double` (*IEEE 754 64 bits*)
 - `boolean` (*1 bit*): `true` or `false`
 - `char` (*16 bits*): *Unicode*

Data Type	Default Value (for fields)
<code>byte</code>	0
<code>short</code>	0
<code>int</code>	0
<code>long</code>	0L
<code>float</code>	0.0f
<code>double</code>	0.0d
<code>char</code>	"\u0000"
String (or any object)	null
<code>boolean</code>	false

Variables

- Allow the storage of typed information or references to objects
- The declaration of a simple variable is performed as follows:

<type> <variable_name> [= <default_value>];

- The variable name can be a set of uppercase or lowercase characters, numbers, or the characters '_' or '\$' (this one should be avoided!)
 - Must not start with a number

Example of typed variables

```
// The number 26, in decimal  
int decVal = 26;
```

```
// The number 26, in hexadecimal  
int hexVal = 0x1a;
```

```
// The number 26, in binary  
int binVal = 0b11010 ;
```

```
double d1 = 123.4;  
// same value as d1, but in scientific notation  
double d2 = 1.234e2;
```

```
float f1 = 123.4f;
```

```
char c = 'A';
```


Example of typed variables

- The '_' can be used to improve the readability of numbers:

```
long creditCardNumber = 1234_5678_9012_3456L ;  
long socialSecurityNumber = 999_99_9999L;
```

```
float pi = 3.14_15F;
```

```
long hexBytes = 0xFF_EC_DE_5E;  
long hexWords = 0xCAFE_BABE;  
long maxLong = 0x7fff_ffff_ffff_ffffL;
```

```
byte nybbles = 0b0010_0101;
```

```
long bytes = 0b11010010_01101001_10010100_10010010;
```

Example of typed variables

... but there are forbidden formats, examples:

```
// Invalid : cannot put underscores adjacent to a decimal point
float pi1 = 3_.1415F;
// Invalid : cannot put underscores adjacent to a decimal point
float pi2 = 3._1415F;
// Invalid : cannot put underscores prior to an L suffix
long socialSecurityNumber1 = 999_99_9999_L;

// OK (literal decimal)
int x1 = 5_2;
// Invalid : cannot put underscores at the end of a literal
int x2 = 52_;
// OK (literal decimal)
int x3 = 5_____2;

// Invalid : cannot put underscores in the 0x radix prefix
int x4 = 0_x52;
// Invalid : cannot put underscores at the beginning of a number
int x5 = 0x_52;
// OK (literal hexadecimal)
int x6 = 0x5_2;
// Invalid : cannot put underscores at the end of a number
int x7 = 0x52_;
```

Conversions between types (cast)

- Numeric type conversions can be performed...
 - Implicitly
 - conversions to types that do not imply any loss of precision
 - `int => long`
 - `int => float`
 - Explicitly
 - conversions to types where some information loss may occur
 - `long => int`
 - `float => int` ("*truncation*")

Wrapper classes

- Since the *Java* language is totally object-oriented, to maintain the compatibility when using primitive data types there is an equivalent defined through an object class type
- *Wrapper* classes:
 - `byte` ⇔ `Byte`
 - `short` ⇔ `Short`
 - `int` ⇔ `Integer`
 - `long` ⇔ `Long`
 - `float` ⇔ `Float`
 - `double` ⇔ `Double`
 - `boolean` ⇔ `Boolean`
 - `char` ⇔ `Character`

Wrapper classes

- These classes provide some functionality that may be useful, for example, to perform conversions

```
int i = Integer.parseInt("1234");
```

```
double d = Double.max(1.0, 2.0);
```

```
boolean b = Character.isUpperCase('Z');
```

```
char c = Character.toLowerCase('Z');
```

```
boolean z = Boolean.getBoolean("true");
```

Boxing and Unboxing

- *Boxing* – conversion of primitive data to the corresponding object
 - `Integer i = 123;`
 - `Integer i = new Integer(123);`
 - `Integer i = Integer.valueOf(123);`
- *Unboxing* – converting instances of *wrapper* classes to the corresponding primitive types
 - Unboxing example:

```
Double d1 = Double.valueOf(123.45);  
double d2 = d1;  
double d3 = d1.doubleValue();
```

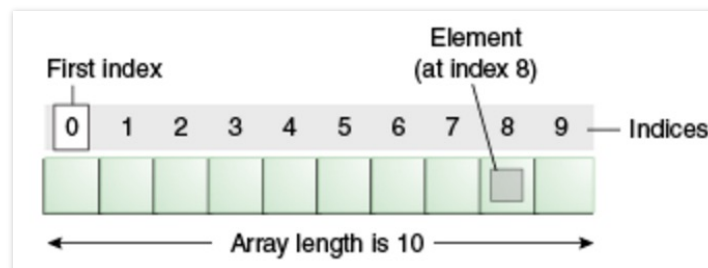
Handling "*Big*" Numbers

- BigInteger
- BigDecimal

```
BigDecimal bd1 = BigDecimal.valueOf(2e-323);  
BigDecimal bd2 = BigDecimal.valueOf(3e-323);  
BigDecimal bd3 = bd1.multiply(bd2);  
System.out.println(bd3); // 6.00E-646
```

Arrays

- Allow the storage of multiple values
 - Support for primitive data or references to objects
 - The maximum number of elements is defined when the *array* is created
 - The *array* has a name, corresponding to the name of the variable that supports it
 - Each value is accessed through the *array* variable and its index ($0..n-1$)



Array declaration

- The declaration of *array variables* can be performed using the following formats

- `<type> [ ] <name>;`
- `<type> <name> [ ];`

- Example:

```
int [ ] ages;
```

```
float months [ ];
```

Array creation

- When an *array* is just declared, the related variable has the value `null`, corresponding to the non-definition of the *array*
- To create (define) the *array* it is necessary to create the number of cells/values intended for it, using the keyword `new`

```
int [ ] ages;
```

```
ages = new int[10];
```

```
float [ ] months = new float[12];
```

Examples of *array* usage

```
ages[0] = 10;
```

```
ages[1] = 15;
```

```
ages[2] = 20;
```

```
int i1 = ages[0] + 1;
```

```
float m = (ages[0]+ages[1])/2.0;
```

Initialization of *arrays*

- When an *array* is created it is possible to initialize it by enumerating the default values
 - *array* dimension is deducted from the number of elements enumerated

```
int    [ ] tab1 = { 10, 20, 30, 40, 50 };  
char   [ ] tab2 = { 'a' , 'b' , 'c' };
```

- The number of elements in an *array* can be easily obtained through the `length` property

```
int n_items = tab1.length;
```

Multidimensional *arrays*

- In Java, *arrays* of multiple dimensions are allowed, which are declared by inserting pairs of [] for each dimension

```
int [ ][ ] array1 = new int[5][2];
```

```
double [ ][ ][ ] array2 = new double[5][4][3];
```

- Items access is carried out as follows

```
array2[0][1][2] = array2[2][1][0] * 3.1415;
```

Multidimensional *arrays*

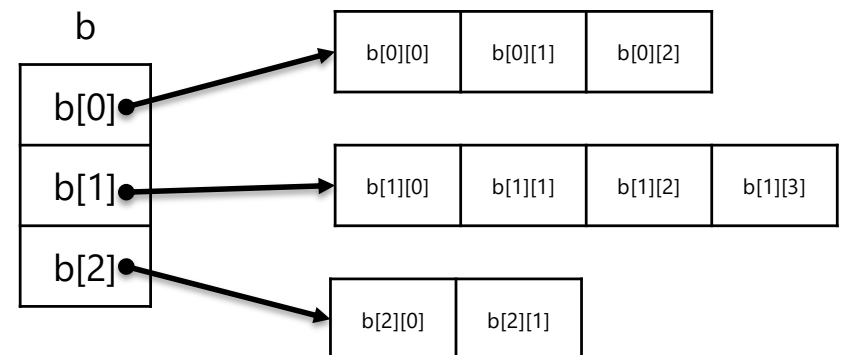
- *Arrays* can have a variable number of elements per line/dimension
 - In this situation, the creation of each line/dimension must be performed in an independent way

```
int [][] b = new int[3][];
```

```
b[0] = new int[3];
```

```
b[1] = new int[4];
```

```
b[2] = new int[2];
```



Iteration over *arrays*

- Getting the number of elements in an *array*

- Assuming `int [][] b = ...`

- `b.length` – number of rows in *array* `b`

- `b[i].length` – number of elements of line *i*

- Example:

```
for ( int i=0 ; i < b.length ; i++)  
    for ( int j=0 ; j< b[i].length ; j++) {  
        // ... b[i][j] ...  
    }
```

Array variables

- Variables of *array type* are references to the corresponding objects
- Taking this into consideration:

```
int [] t1 = {1,2,3};  
int [] t2;  
t2 = t1;  
t2[1] = 123;  
System.out.println(t1[1]); //123
```


Utility methods

- There are several classes and libraries in Java that provide useful methods for different situations
- For operations on *arrays*, the following can be highlighted:
 - `System.arraycopy(src, src_pos, dst, dst_pos, length)`
 - Allows to copy elements from one *array* to another, previously created
 - The `java.util.Arrays` class
 - Provides a set of methods for working with *arrays*
 - `copyOf`, `copyOfRange`
 - `fill`
 - `compare`, `equals`, `mismatch`
 - `binarySearch`, `sort`, `stream`
 - `toString`

Operators

- The operators are similar to those existing in other languages (C/C++, C#,...)

Operator Precedence	
Operators	Precedence
postfix	<i>expr</i> ++ <i>expr</i> --
unary	++ <i>expr</i> -- <i>expr</i> + <i>expr</i> - <i>expr</i> ~ !
multiplicative	* / %
additive	+ -
shift	<< >> >>>
relational	< > <= >= instanceof
equality	== !=
bitwise AND	&
bitwise exclusive OR	^
bitwise inclusive OR	
logical AND	&&
logical OR	
ternary	? :
assignment	= += -= *= /= %= &= ^= = <<= >>= >>>=

Exception

assignment operator =

- Different behaviors depending on the data type:
 - primitive data
 - value is copied
 - objects
 - Variables only store the reference to the object
 - **a copy of the reference is made**
 - The object referenced by both variables is the same
- Passing parameters when calling a function follows the same logic as the assignment operator

Garbage Collector

- When operations like assignment, parameter passing or similar are executed with references to objects, the number of references to the related instances is automatically managed centrally
- When the number of references is zero it means that there is no way to access the object and, as such, the object is marked for deletion
 - It is not the programmer's responsibility to free the associated memory

Garbage Collector

- The management of the memory that will be reserved and its subsequent release is done by the *Garbage Collector*
 - When objects are marked for deletion the `finalize()` method is invoked
 - There is no guarantee when this method is called
 - Marked as *deprecated* since version 9
 - After the `finalize` method is called, the memory associated with the object will be freed, but there is no guarantee of when

`==` and `!=` operators

- When the operators `==` and `!=` are used with reference-type operands, they check whether the two operands refer to the same object

```
Integer a1 = new Integer(123);  
Integer a2 = new Integer(123);  
Integer a3 = a1;  
System.out.println(a1==a2);           // false  
System.out.println(a1==a3);           // true  
System.out.println(a2==a3);           // false  
System.out.println(a1.equals(a2));    // true
```

instanceof

- The `instanceof` operator allows checking whether a given reference refers to an object of a specified type

```
if (value1 instanceof Integer) { // Before Java 16
    valuei = (Integer) value1;
    ...
}
```

```
if (value1 instanceof Integer valuei) { // Java 16+
    // valuei is of type Integer
    ...
}
```

Flow control – if

- *if-then*

```
if (condition) {  
    ...  
}
```

- *if-then-else*

```
if (condition) {  
    ...  
} else {  
    ...  
}
```

```
if (condition1) {  
    ...  
} else if (condition2) {  
    ...  
} else if (condition3) {  
    ...  
} else {  
    ...  
}
```


Flow control – switch, break

- *switch-case* + *break*

```
switch (source) {  
    case op1[,op2]:  
        [case op3:]  
            ...  
            break;  
    ...  
    default :  
        ...  
        break;  
}
```

- Strings can be used in cases
- Can be used as an expression

- *switch-arrow case* (Java SE 14)

```
switch (source) {  
    case op1[,op2] -> ... ;  
    case op3 -> ... ;  
    ...  
    default -> ... ;  
}
```

- Similar to classic *switch-case*, but the instruction `break` is not needed

Switch examples

```
var v1 = switch(value) {  
    case 1 -> "One";  
    case 2 -> "Two";  
    case 3 -> "Three";  
    default -> "Other";  
};
```

```
var v2 = switch(value) {  
    case 1:  
        yield 10;  
    case 2,3:  
        yield 25;  
    default:  
        yield 50;  
};
```

```
var v3 = switch(value) {  
    case 1 -> {  
        System.out.println("v3.1");  
        yield 0b01010;  
    }  
    case 2 -> {  
        System.out.println("v3.2");  
        yield 0x014;  
    }  
    case 3,5 -> {  
        System.out.println("v3.3/5");  
        yield 036;  
    }  
    default -> {  
        System.out.println("v3.def");  
        yield 3.14159f;  
    }  
};
```

Flow control – cycles

- *for*

```
for(<init>; <condition>; <update>) {  
    ...  
}
```

- *for each*

```
for(<type> <var>:<collection/array>) {  
    ...  
}
```

- *while*

```
while (condition) {  
    ...  
}
```

- *do-while*

```
do {  
    ...  
} while (condition);
```

flow control

- Other flow control statements in the execution of a Java program:
 - `continue`
 - `break`
 - The `continue` and the `break` instructions can be followed by a *label* associated with the loop in which they are related to

```
loop1: for(int i=0; i<10 ; i++)  
        for(int j=10; j>0 ; j--)  
            if (i == j) break loop1;
```

- `return`

User interaction

- *Java* provides a set of objects that allow representing the typical inputs and outputs of a program/process
 - `System.in`
 - Represents the default input, typically associated with console input
 - `System.out`
 - Represents the default output, typically associated with output to the console
 - `System.err`
 - Represents output for errors, typically associated with output to the console

Data output

- `System.out.print(...)`
- `System.out.println(...)`
- `System.out.printf(<format>, param1, param2, ...)`

- `System.err.print(...)`
- `System.err.println(...)`
- `System.err.printf(<format>, param1, param2, ...)`

Data input

- Although the `System.in` object allows obtaining data entered by the user in the console, this data is interpreted as *bytes*
- To facilitate access to this information, in an easier and more typified way, a `Scanner` object can be used

```
Scanner sc = new Scanner(System.in);
```

- `Scanner` object allows to read character sequences separated by delimiters

```
int n = sc.nextInt();           // Read an integer  
double x = sc.nextDouble();    // Read a double
```

Data input

- By default, the delimiters are *whitespace*, *tabs* and *newlines*. However, other delimiters can be defined:

```
sc.useDelimiter("-");
```

- The type of the next value to be read can be tested using:

```
// Check if the next value is a double
if ( sc.hasNextDouble() )
    x = sc.nextDouble();
```


Math

- `Math` class, belonging to the `java.lang` *package*, provides a set of mathematical functions
 - `sin`, `cos`, `tan`
 - `abs`, `round`, `floor`, `ceil`, `rint`
 - `max`, `min`
 - `pow`, `exp`
 - `hypot`, `sqrt`
 - `random`
 - `toDegrees`, `toRadians`
 - `E`, `PI`

Math.random

- Method that generates *pseudo-random numbers*

```
double r = Math.random();
```

- $0.0 \leq r < 1.0$

- Example for generating numbers between 1 and 100, inclusive

```
int i = (int) (Math.random() * 100) + 1;
```

Class Random

- In the *package* `java.util`, the `Random` class is provided, which has a set of methods that allows a more flexible access to sequences of *pseudo-random numbers*

```
Random rnd = new Random();  
int i = rnd.nextInt();  
int j = rnd.nextInt(100) + 1;  
double d = rnd.nextDouble();  
...  
rnd.setSeed(1234);  
...
```

String

- The `String` class represents an *immutable char sequence*
 - Changing a `String` (for example, concatenating using the '+' operator) always creates a new `String`
- This class provides utility methods to work with the characters present in a `String`
 - `equals`, `equalsIgnoreCase`, `matches`, `compareTo`, `compareToIgnoreCase`, `startsWith`, `contains`, `endsWith`, `indexOf`
 - `isBlank`, `isEmpty`
 - `concat`, `replace`, `replaceAll`, `repeat`, `trim`
 - `split`
 - `toUpperCase`, `toLowerCase`
 - ...

String

- There are some cautions about the `String` class to point out, which can be verified by the following example

```
String s1 = "DEIS-ISEC";  
String s2 = new String("DEIS-ISEC");  
String s3 = s2;  
String s4 = "Deis-Isec";  
  
System.out.println(s1 == s2); // false  
System.out.println(s2 == s3); // true  
s3 = s1;  
System.out.println(s2 == s3); // false  
System.out.println(s1.equals(s2)); // true  
System.out.println(s2.equals(s4)); // false  
System.out.println(s2.equalsIgnoreCase(s4)); // true
```

Text blocks

- Since *Java 15*, it is possible to define text blocks (multiline strings)

```
String s1 =  
    " Begin\n"+  
    "   Hello\n"+  
    " End";
```

can be written as:

```
String s1 = """  
    Begin  
        Hello  
    End  
"""; // the leftmost whitespace determines the indentation
```

StringBuffer and StringBuilder

- The `StringBuffer` and `StringBuilder` classes allow managing *mutable strings*
- Both classes provide several methods for manipulating *strings*, including methods for adding new characters or *strings*, modifying individual characters or *substrings*
- Main differences:
 - `StringBuffer` – *thread-safe*
 - `StringBuilder` – *faster*

```
StringBuilder sb = new StringBuilder();  
sb.append("Hello, ");  
sb.append("world!");  
System.out.println(sb.toString());
```

Java language

Object oriented programming

Object-oriented programming

- Being a fully object-oriented language, *Java* supports the key principles of object-oriented programming, including:
 - Abstraction and Encapsulation
 - Inheritance
 - Polymorphism

Object-oriented programming

- Being a fully object-oriented language, *Java* supports the key principles of object-oriented programming, including:
 - Abstraction and Encapsulation
 - Define new object types

```
class <new_class_name> { ... }
```
 - Hide implementation details
 - Control access to information
 - Generalize the way it is used
 - Inheritance
 - Polymorphism

Object-oriented programming

- Being a fully object-oriented language, *Java* supports the key principles of object-oriented programming, including:
 - Abstraction and Encapsulation
 - Inheritance
 - Defining a new object type as a specialization of another object type

```
class < new_class > extends < base_class > { ... }
```
 - The characteristics of the base object are inherited
 - New characteristics and behaviors of the specialization can be defined
 - Polymorphism

Object-oriented programming

- Being a fully object-oriented language, *Java* supports the key principles of object-oriented programming, including:
 - Abstraction and Encapsulation
 - Inheritance
 - Polymorphism
 - Redefining (`@Override`) the behaviors declared and/or defined in the base class

Classes

- To define a new class of objects, the following syntax is used:

```
class <name> {  
    <variables>  
    <methods/functions>  
}
```

- While it is not mandatory to define all variables at the beginning or all methods at the end, maintaining this order is considered good practice for organization

Classes

- Example:

```
class Point {  
    int x,y;  
  
    void move(int dx, int dy) {  
        x += dx;  
        y += dy;  
    }  
}
```

- To create an instance of the class, which is commonly referred to as an object, the `new` keyword is used:

```
Point p = new Point();
```

Method *overload*

- Multiple methods can have the same name, as long as they can be distinguished by their parameters
- The distinction can be made based on:
 - Number of parameters
 - Parameter type
- The return type of a method is not used to distinguish between *overloaded* methods

Constructors

- Whenever an object is created, a constructor is automatically called to initialize the object
- Constructors are methods with the same name as the class and no return type declared
- Multiple constructors can be defined, with different parameters
 - A constructor without parameters is called *default constructor*

Constructors

- In Java, variables that are not explicitly initialized (either in the constructor or by direct assignment when declared) are initialized with default values (0 for numeric types, `false` for boolean, `null` for reference types)

```
class Point {  
    int x, y;  
    int z; // by default, it will be 0  
  
    Point( int xi, int yi ) {  
        x = xi;  
        y = yi;  
    }  
}
```

this

- In the context of an instance of a particular object type, its reference can be accessed (self-reference) using the keyword `this`

```
class Point {  
    int x, y;  
  
    Point( int x, int y ) {  
        this.x = x;  
        this.y = y;  
    }  
    // ...  
}
```

Constructors delegation

- A constructor can delegate the creation of an object to another constructor of the same class (when there are overloaded constructors)
 - Delegation is done by using `this` as if it were a function, with the parameters being those of the constructor to which the object creation will be delegated

```
class Point {  
    int x, y;  
  
    Point() { this(0,0); }  
  
    Point( int x, int y ) {  
        this.x = x;  
        this.y = y;  
    }  
}
```

Static members

- In the definition of a class, the various members, variables or functions, can be defined as being *static*
 - The `static` tag is used
- The static members...
 - Can be accessed/used without instances of this type of object
 - Its use is made by indicating the name of the class itself and not a reference to an object of that type
 - The values of the static variables are accessed and subject to change, being shared by any existing instances of this type of object
 - Changes made to static variables in the context of one instance are visible to all other instances

Static members

- Static methods can only call other static methods
 - They can call instance methods through they are properly scoped to their respective instances
- In the context of static members, the `this` reference is not used, since static members are not associated with any instance
- Static variables are only initialized on the first usage of the class
 - Values can be initiated by direct assignments or via a *static constructor*
 - Defined through: `static { ... }`

Static members

```
class Person {
    static int count;
    static {
        count = 101;
    }
    static void resetCounter() {
        count = 1;
    }

    int id;
    String name;

    Person( String name ) {
        id = (count++);
        this.name = name ;
    }
    void show() {
        System.out.printf("%5d - %s\n", id, name);
    }
}
```

```
public class Main {
    public static void main( String [] args ) {
        Person p1 = new Person("João Felix");
        Person p2 = new Person("Cristiano Ronaldo");
        p1.show();
        p2.show();
        Person.resetCounter();
        var p3 = new Person("Eusébio Ferreira");
        p3.show();
    }
}
```

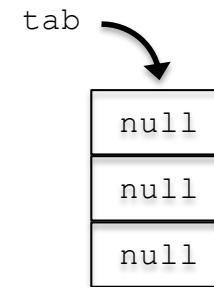
Output:

```
101 - João Felix
102 - Cristiano Ronaldo
1 - Eusébio Ferreira
```

Object *arrays*

- An *array* of objects, when created without explicitly specifying the elements, contains only `null` references

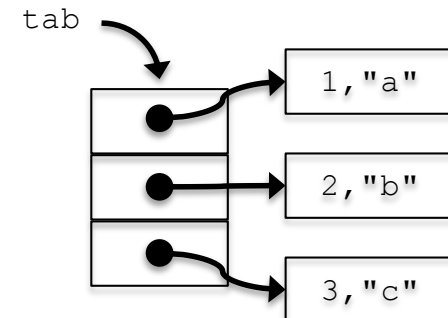
```
Person [] tab = new Person[3];
```



Object *arrays*

- An *array* of objects, when created without explicitly specifying the elements, contains only `null` references

```
Person [] tab = new Person[3];  
tab[0] = new Person("a");  
tab[1] = new Person("b");  
tab[2] = new Person("c");
```



Object *arrays*

- Object *arrays* can be created by enumeration

```
Person [] tab = { new Person("a"), new Person("b") };
for(var p : tab)
    p.show();
Person [][] groups = {
    { new Person("a1"), new Person("a2") },
    { new Person("a3"), new Person("a4") }
};
for(var group : groups) {
    for(var student : group)
        System.out.print("\t" + student.name);
    System.out.println();
}
```

- When it is necessary to pass an *array* of objects created by enumeration to a function, the following format can be used:

```
proc( new Person [] { new Person("a"), new Person("b") } );
```

Packages

- Classes are grouped into *packages*
 - For code organization
 - Creation of *namespaces* to name the classes
- Solve name conflicts between different classes
 - When there is conflict of names between classes of different *packages*, its full name must be indicated, ie, including the name of the package: `<package>.<class>`
- Simplify application distribution
 - Easier way to make Java programs available for use by end users
- Facilitate code reusability
 - Many features can be reused in other projects through the constitution and availability of *packages* with object classes
 - It follows the same logic of using all the classes that are available in the *Java API*

Packages – creation

- *Package* definition
 - The *package* has a name defined through the statement `"package <name>;"` indicated as the first effective line in a code file
 - The base name is usually made up of two separate pieces of information, in lowercase
 - Company domain name in reverse order (`deis.isec.pt => pt.isec.deis`)
 - Project name
 - Example: `package pt.isec.a200212345.proj_lesson1;`
 - Files that include classes belonging to a *package* must be placed in a directory hierarchy corresponding to the *package* name

```
Project_Lesson1
  pt
    isec
      a200212345
        project_lesson1
          Example1.java
```

Packages – usage

- To use classes or other elements defined in different *packages*, either the full name or an `import` statement for the respective element can be used
 - There may be multiple `import` statements (usual!)
- `import` usage examples:
 - All elements of a package will be included:

```
import pt.isec.pa.utils.*;
```
 - Only the desired element will be included:

```
import pt.isec.pa.utils.FileUtils;
```

Packages – example

```
PA@deis$ cat Example1.java
```

```
package pt.isec.pa.example1;
```

```
public class Example1 {  
    public static void main(String args[]) {  
        System.out.println("Java@DEIS-ISEC");  
    }  
}
```

```
PA@deis$ javac Example1.java
```

```
PA@deis$ java Example1
```

```
Error: Could not find or load main class Example1
```

```
Caused by: java.lang.NoClassDefFoundError: pt/isec/pa/example1/Example1  
(wrong name: Example1)
```

```
PA@deis$ mkdir -p pt/isec/pa/example1
```

```
PA@deis$ mv Example1.class pt/isec/pa/example1
```

```
PA@deis$ java pt.isec.pa.example1.Example1
```

```
Java@DEIS-ISEC
```

```
PA@deis$
```

Packages – example

```
PA@deis$ cat Example1.java
```

```
package pt.isec.pa.example1;
```

```
public class Example1 {  
    public static void main(String args[]) {  
        System.out.println("Java@DEIS-ISEC");  
    }  
}
```

```
PA@deis$ javac -d . Example1.java
```

```
PA@deis$ java pt.isec.pa.example1.Example1
```

```
Java@DEIS-ISEC
```

```
PA@deis$
```

Packages – access

- In addition to the organizational and reusability benefits that *packages* provide, they also allow control over access to the implementation details of their elements
- Access control for different members is managed using the keywords `public`, `protected`, `private`, or by omitting a modifier (*no label*), following the rules outlined in the table below

Tag	Class	Package	Subclass	Others
<code>public</code>	Yes	Yes	Yes	Yes
<code>protected</code>	Yes	Yes	Yes	No
<i>(no label)</i>	Yes	Yes	No	No
<code>private</code>	Yes	No	No	No

- Note: The class definition statement only allows the `public` modifier or no modifier (however, nested classes can have other modifiers)

jar files

- To facilitate the deployment of *Java* programs, all classes that compose the program, along with other resources (such as images and sounds), must be included in a `jar` file
 - A `jar` file is essentially a `zip` archive
 - It simplifies the maintenance of the directory hierarchy representing the various *packages* that constitute a program
 - Additionally, a *manifest file* is included, allowing the specification of special attributes

jar files

- Create a `jar` file

```
jar -cf example1.jar pt/*
```

- Run a `jar` file

```
java -cp example1.jar pt.isec.pa.example1.Example1
```

Executable jar files

- When creating a jar file you can specify a class that includes a main function, which will be executed when the jar file is executed with: `java -jar <file.jar>`

```
PA@deis$ javac -d . Example1.java
PA@deis$ jar -cfe example1.jar pt.isec.pa.example1.Example1 pt/*
PA@deis$ java -jar example1.jar
Java@DEIS-ISEC
PA@deis$ mkdir temp
PA@deis$ cp example1.jar temp/example1.jar
PA@deis$ cd temp
PA@deis$ unzip example1.jar
Archive:  example1.jar
   creating: META-INF/
  inflating: META-INF/MANIFEST.MF
   creating: pt/isec/
   creating: pt/isec/pa/
   creating: pt/isec/pa/example1/
  inflating: pt/isec/pa/example1/Example1.class
PA@deis$ cat META-INF/MANIFEST.MF
Manifest-Version: 1.0
Created-By: 17.0.2 (Oracle Corporation)
Main-Class: pt.isec.pa.example1.Example1
```

Manifest file

- The *manifest file*, created automatically in the previous example, can be created and configured manually

```
PA@deis$ cat > MANIFEST.TXT
Main-Class: pt.isec.pa.example1.Example1
PA@deis$ jar -cfm example1.jar MANIFEST.TXT pt/*
PA@deis$ java -jar example1.jar
Java@DEIS-ISEC
PA@deis$ mkdir temp
PA@deis$ cp example1.jar temp/example1.jar
PA@deis$ cd temp
PA@deis$ unzip example1.jar
Archive:  example1.jar
  creating: META-INF/
  inflating: META-INF/MANIFEST.MF
    creating: pt/isec/
    creating: pt/isec/pa/
    creating: pt/isec/pa/example1/
  inflating: pt/isec/pa/example1/Example1.class
PA@deis$ cat META-INF/MANIFEST.MF
Manifest-Version: 1.0
Main-Class: pt.isec.pa.example1.Example1
Created-By: 17.0.2 (Oracle Corporation)
```

Composition and Inheritance

- A class, on its own, represents a specific concept
- In most cases, the concept is defined through characteristics or properties, which are represented using variables

Composition and Inheritance

- Depending on complexity and specific requirements, an object's definition may follow different models:
 - *Composition*
 - *Inheritance*
 - A combination of both
- These techniques allow a new object to take advantage of the characteristics and behaviors of other objects without requiring their reimplementation

Composition

- *Composition*
 - An object is defined as an aggregation of multiple objects
 - For example, a computer consists of a motherboard, a processor, a graphics card, and other components

```
class Student {  
    long nr;  
    String name;  
  
    public Student( long nr, String name ) {  
        this.nr = nr;  
        this.name = name;  
    }  
    // ...  
}  
  
class Group {  
    Student s1;  
    Student s2;  
  
    public Group( Student s1, Student s2 ) {  
        this.s1 = s1;  
        this.s2 = s2;  
    }  
    // ...  
}
```

Inheritance

- *Inheritance*
 - An object is defined as a specialization of a more generic object
 - All characteristics that define a base object type are inherited, and only the differences from the base object need to be implemented
 - For example, a car with automatic transmission is still a car but with a different gear-shifting mechanism, so only this behavior may need to be redefined

Inheritance

- To create a new class of objects from another one, inheriting all its characteristics, the following syntax is used:

```
class < new_class > extends < base_class > {  
    . . .  
}
```


Inheritance

- With no additional information or behavior defined, the derived class instances are functionally identical to the base class
 - The derived class is also called a *subclass*
- ***In Java a class can only inherit from one other class (single inheritance)***
 - Multiple inheritance is not allowed

Inheritance

- In the context of a subclass the features of the base class can be referred using the keyword `super`
 - `super.<base_class_variable>`
 - `super.<base_class_method>([<params, ...>]);`

Inheritance

- A derived class has access to the `public` and `protected` methods and variables of the base class
 - It cannot access members marked as `private` in the base class
 - *Package-private* (untagged) members remain accessible if the derived class belongs to the same package

Inheritance

- If no explicit constructor is provided in the derived class, the *default constructor* of the base class is called automatically, unless the base class does not have one
- If the base class does not have a *default constructor*, the subclass must define a constructor that explicitly calls a base class constructor
 - Parameters are passed to the base class constructor by using a call to the `super` function with the respective parameters in the first line of the subclass constructor
 - This process is similar to delegating between constructors of the same class, but replaces `this` with `super`

Inheritance

- Behaviors (*methods!*) defined in base classes can be redefined in derived classes
 - The access level cannot be more restricted than defined in the base class
 - If the base class defines a method as `public`, in the subclass it cannot be redefined as `protected`
 - If the base class defines a method as `protected`, in the subclass it can be redefined as `protected` or `public`
- The redefined methods must have the annotation `@Override`

```
@Override  
void procData() { ... }
```

Inheritance – Example

```
class Student {
    protected long nr;
    protected String name;

    public Student( long nr, String name ) {
        this.nr = nr;
        this.name = name;
    }

    public long getNr() {
        return nr;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return String.format("a%d@isec.pt", nr );
    }
    // ...
}
```

Inheritance – Example (cont .)

```
class ErasmusStudent extends Student {  
    protected String country;  
  
    public ErasmusStudent( long nr, String name, String country) {  
        super( nr, name );  
        this.country = country;  
    }  
  
    public String getCountry() {  
        return country;  
    }  
  
    @Override  
    public String getEmail () {  
        return String.format("a%d.%s@isec.pt", nr, country );  
    }  
  
    // ...  
}
```

Inheritance – Example (cont .)

```
public class Main {  
    public static void main( String [] args ) {  
        ErasmusStudent e1 = new ErasmusStudent(20220202021,"John Smith","uk");  
        ErasmusStudent e2 = new ErasmusStudent(20220303031,"Pierre Durand","fr");  
        ErasmusStudent [] tab = {e1, e2};  
        for(ErasmusStudent s: tab)  
            System.out.printf("%s: %s\n", s.getName(), s.getEmail());  
    }  
}
```

Output:

```
John Smith: a2022020202.uk@isec.pt  
Pierre Durand: a2022030303.fr@isec.pt
```


final

- The `final` keyword indicates that the element where it is applied is not subject to change
- When applied to a...
 - variable
 - E.g.: `final int i = 123;`
 - When a `final` variable is assigned with a value, that value cannot be changed.
 - If this value is a reference to an object, the internal characteristics of that object can be changed as long as they are not `final`
 - A `final` variable will have to get a value, by direct assignment or get its value in the context of all the class constructors (in the case of member variables)
 - method
 - E.g.: `public final String getEmail() { ... }`
 - When the method is marked `final` then it cannot be overridden in derived classes
 - class
 - E.g.: `final class ErasmusStudent extends Student { ... }`
 - A class marked `final` cannot be the base class for another class.
 - In other words, "*cuts the inheritance process*"

Single hierarchy – the `Object` class

- In Java all classes derive directly or indirectly from the `Object` class
 - Even when this dependence is not made explicit, it is assumed implicitly

```
class MyClass { ... }
```

`<=>`

```
class MyClass extends Object { ... }
```

Single hierarchy – the `Object` class

- The `Object` class provides a set of methods, which include:
 - `clone`
 - `equals`
 - `getClass`
 - `hashCode`
 - `toString`

Object.toString()

- The `public String toString()` method should return a `String` object that represents the object instance to which it applies
- By default (if not overridden by a derived class) the full name of the class is returned followed by a hexadecimal value representing the instance (corresponding to the object's *hash code*)
 - `MyObject@1fa4d5`

`Object.equals (Object)`

- An object that is going to be compared with other similar objects must provide an implementation of the `equals` method, inherited from the `Object` class
- The `public boolean equals (Object other)` method receives the reference to the other object as a parameter with which the comparison is intended, and must return...
 - `true`, if considered equal
 - `false`, if considered unequal

Object.equals (Object)

```
@Override  
public boolean equals (Object o) {  
    // ...  
    return <comparison_result>;  
}
```

- Keep in mind that the `equals` method parameter is of type `Object`, that is, it can reference any type of Java object

Object.equals (Object)

- The `equals` method should check if the comparison is being performed with the appropriate object, alternatively...
 - Force it to be exactly one object of the same type

```
this.getClass () == other.getClass ()
```
 - Allow the object to be an instance of an object type of a certain (sub)hierarchy, usually, the same to which the object itself belongs

```
other instanceof <base_class>
```
- In addition to verifying the type, the content to be compared must be verified
 - Need to cast to the desired type to access proper members

Object.hashCode()

- Whenever the `equals` method is redefined, the `hashCode` method must also be redefined
- The `public int hashCode()` method should return an integer representing the instance in question
 - It can correspond to a unique identifier assigned to the object or it can be calculated taking into account one or more characteristics of the object
 - Usually, the same characteristics that are used for the comparison in `equals` should be considered
 - By default (implementation of the `Object` class) returns an integer value corresponding to its internal address

```
@Override
public int hashCode() {
    return <unique_value_for_the_instance>;
}
```


Object.hashCode()

- Important:
 - Two objects considered equal by the `equals` method must have the same *hash code*
 - It is not necessary for two objects considered different by `equals` to have different *hash codes*
- the *hash code* is used to optimize information indexing (similar to *clustering techniques*), for example in the context of data collections
 - The search for similar elements is only performed in groups of items (*clusters*) that have the same *hash code*
 - If two identical objects have *different hash codes* (causing them to be placed in different *clusters/groups*) the desired objects will not be found

equals() and hashCode()

```
class PersonA {
    String name;
    int age;

    public PersonA(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass())
            return false;

        PersonA personA = (PersonA) o;

        return Objects.equals(name, personA.name);
    }

    @Override
    public int hashCode() {
        return name != null ? name.hashCode() : 0;
    }

    @Override
    public String toString() {
        return String.format("%s [%d]", name, age);
    }
}
```

```
class PersonB extends PersonA {
    public PersonB(String name, int age) {
        super(name, age);
    }

    @Override
    public int hashCode() {
        return (int) (Math.random()*100);
    }
}

class PersonC extends PersonA {
    public PersonC(String name, int age) {
        super(name, age);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o instanceof PersonA person) {
            return Objects.equals(name, person.name);
        }
        return false;
    }
}
```

equals () and hashCode ()

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Equals and Hashcode");  
        Set<PersonA> setA = new HashSet<>();  
        setA.add(new PersonA("aaaa",10));  
        setA.add(new PersonA("aaaa",10));  
        setA.add(new PersonA("bbbb",10));  
        setA.add(new PersonA("aaaa",20));  
        for (var person: setA)  
            System.out.println(person.toString());  
  
        System.out.println();  
  
        Set<PersonA> setB = new HashSet<>();  
        setB.add(new PersonB("aaaa",10));  
        setB.add(new PersonB("aaaa",10));  
        setB.add(new PersonB("bbbb",10));  
        setB.add(new PersonB("aaaa",20));  
        for (var person: setB)  
            System.out.println(person.toString());  
  
        System.out.println();  
  
        PersonA a=new PersonA("abc",123);  
        PersonB b=new PersonB("abc",123);  
        PersonC c=new PersonC("abc",123);  
        System.out.println(a.equals(b));  
        System.out.println(b.equals(a));  
        System.out.println(a.equals(c));  
        System.out.println(c.equals(a));  
        System.out.println(b.equals(c));  
        System.out.println(c.equals(b));  
    }  
}
```

Output example:

```
Equals and Hashcode  
aaaa [10]  
bbbb [10]  
  
bbbb [10]  
aaaa [20]  
aaaa [10]  
aaaa [10]  
  
false  
false  
false  
true  
false  
true
```

Object.clone()

- The protected method `Object clone()` should return an exact copy of the instance in the context it is called from
- To create *clones* of objects of a certain class using this mechanism, the class must implement the `Cloneable` interface
 - As the `Object` class does not implement the `Cloneable` interface, it generates an exception that will cause the program to be interrupted:
 - `CloneNotSupportedException`

Object.clone()

- The redefinition of this method in a derived class should call the `clone` method of the base class:

```
super.clone()
```

- A low-level copy is made
 - Object type constructor is not called
 - Additional changes may be required after calling the base class `clone`, as the object may contain references to objects, which must also be *cloned*
- Note: To create clones a class can offer a *copy constructor*. Using this technique, the call and execution of the constructor is automatically assured

Object.clone()

```
class ClassA implements Cloneable {
    int mult;
    MyNumber nr;
    ClassA(int mult,int i) {
        this.mult = mult;
        nr = new MyNumber(i);
    }
    void inc() {nr.setI(nr.getI()+1);}
    void show() { System.out.println(nr.getI()*mult); }

    @Override
    protected ClassA clone() throws CloneNotSupportedException {
        return (ClassA) super.clone();
    }
}

class ClassB implements Cloneable {
    int mult;
    MyNumber nr;
    ClassB(int mult,int i) {
        this.mult = mult;
        nr = new MyNumber(i);
    }
    void inc() {nr.setI(nr.getI()+1);}
    void show() { System.out.println(nr.getI()*mult); }

    @Override
    protected ClassB clone() throws CloneNotSupportedException {
        ClassB newObj=(ClassB) super.clone();
        newObj.nr = new MyNumber(nr.getI());
        return newObj;
    }
}
```

```
class MyNumber {
    public int i;
    public MyNumber(int i) { this.i = i; }
    public int getI() { return i; }
    public void setI(int i) { this.i = i; }
}

public class Main {
    public static void main(String[] args)
        throws CloneNotSupportedException {
        System.out.println("Clone");
        ClassA a1 = new ClassA(10,7);
        a1.inc();
        ClassA a2 = a1.clone();
        a2.inc();
        a1.show();
        a2.show();
        System.out.println();

        ClassB b1 = new ClassB(10,7);
        b1.inc();
        ClassB b2 = b1.clone();
        b2.inc();
        b1.show();
        b2.show();
    }
}
```

Output:

```
Clone
90
90

80
90
```

Object.getClass()

- The `Class<?> getClass()` method returns a `Class` object representative of the type of object to which the instance corresponds
- A `Class` object allows a program to take advantage of "introspection" and "reflection" functionalities
 - It allows to get information about the code itself (constructors, methods, variables, annotations, ...)
 - For example, having the reference to a `Class` object, corresponding to a certain type of object, it is possible to create new instances of that type, enumerating the respective constructors and executing the desired one
- Comparing instances of `Class` objects permits checking whether two objects are of exactly the same type
 - `obj1.getClass() == obj2.getClass()`

Object.getClass()

```
class ClassA {
    public ClassA() {}
    protected ClassA(int i) {}
    private void methodA() {}
    protected void methodB() {}
    public void methodC() {}
}

public class Main {
    public static void main(String[] args) {
        System.out.println("GetClass Example");
        ClassA obj = new ClassA();
        Class type1 = obj.getClass();
        Class type2 = ClassA.class;
        System.out.println(type1.equals(type2)); //true
        for(var constructor:type1.getDeclaredConstructors())
            System.out.printf("Constructor: %d %s\n",
                               constructor.getModifiers(),
                               Arrays.toString(constructor.getParameters())
                               );
        System.out.println();
        for(var method:type1.getMethods())
            System.out.printf("Method '%s': %d\n",
                               method.getName(),
                               method.getModifiers()
                               );
        System.out.println();
        for(var method:type1.getDeclaredMethods())
            System.out.printf("Method '%s': %d\n",
                               method.getName(),
                               method.getModifiers()
                               );
    }
}
```

Output:

```
GetClass Example
true
Constructor: 1 []
Constructor: 4 [int arg0]
```

```
Method 'methodC': 1
Method 'equals': 1
Method 'toString': 1
Method 'hashCode': 257
Method 'getClass': 273
Method 'notify': 273
Method 'notifyAll': 273
Method 'wait': 17
Method 'wait': 17
Method 'wait': 17
```

```
Method 'methodA': 2
Method 'methodB': 4
Method 'methodC': 1
```


sealed (Java 17)

- A *sealed class* only allows as subclasses those that are indicated with their definition

```
sealed class < base_class > permits <other1>, <other2>,  
... {  
    // ...  
}
```

- A *class* derived from a *sealed class* must be `final`, `sealed` or `non-sealed`

- Example:

```
sealed class A permits B,D,E { /* ... */ }  
sealed class B extends A permits C { /* ... */ }  
final class C extends B { /* ... */ }  
non-sealed class D extends A { /* ... */ }  
final class E extends A { /* ... */ }  
class F extends D { /* ... */ }
```

Advanced Programming

Abstract classes

Interfaces

Abstract classes and methods

- A *class* may not provide implementation for all of its methods or simply not allow direct instances to be created
 - For example, classes that represent abstract concepts
- The methods that are only declared, i.e., do not have a *scope*, must be tagged with the keyword `abstract`
- Classes that have *abstract methods* must also be classified as abstract, and must include the `abstract` tag
- A class can be declared abstract even if it doesn't have any *abstract methods*

Abstract classes and methods

- Classes that derive from *abstract classes* must provide implementations for the *abstract methods* or be classified as abstract as well
- An abstract class cannot be instantiated, but variables of the *abstract class* type can reference instances of classes that are derived from it

Abstract classes and methods

```
abstract class Fruit {
    // ...
    abstract int getColor();
    // ...
}

class Orange extends Fruit {
    // ...
    @Override
    int getColor() {
        return 0xFFA500;
    }
    // ...
}

...
//Fruit f_error = new Fruit(); // Error!
Orange o = new Orange();
Fruit f = new Orange();
```

Interfaces

- At its limit, an *abstract class* can be constituted by a set of methods that are all abstract, without any associated functionality
- The *Java* language provides a more adequate form for this type of declaration, which is called an *interface*

Interface

- An *interface* in *Java* is a description of a protocol that other *interfaces* or classes have to respect
- In their most traditional use, an *interface* allows the declaration of a set of methods that one or more classes will have to respect

Interface

- *Interface* definition example:

```
interface MyInterface {  
    void my_func();  
}
```

- Implementing an *interface*:

```
class MyClass implements MyInterface {  
    @Override  
    public void my_func() {  
        // ...  
    }  
}
```


Interface

- All implemented methods are `public`
 - Cannot restrict access to be `protected` or `private`
- A class can implement several *interfaces*
 - Remember: a class can only derive from another single class (only one!)
- Notes:
 - If an *abstract class* is used instead of an *interface*, `protected` methods can be defined to be overridden in derived classes
 - Interfaces can be defined as *sealed*, just like classes

Interface

- A *class* that implements an *interface*, which does not provide implementations for *abstract methods*, is considered abstract and must be labelled with `abstract`
- A variable of an *interface*'s type can reference instances of classes that implement that *interface*
 - However, with a variable of an *interface* type, it can only access the functionalities (methods) declared in that *interface*
- Note: In the previous example, an instance of `MyClass` is also an instance of `MyInterface`
 - i.e., `<object> instanceof MyInterface` will return `true`

Interface

- In addition to declaring methods that classes must implement, an *interface* can also contain:
 - *constants*
 - *default methods*
 - *static methods*

Interface – constants

- *Interface constants*
 - Variables that are defined in the *interface*
 - These variables automatically receive the "modifiers" `public`, `static` and `final`, in other words, they are considered *constants*

```
interface InterfaceA {  
    int MAX_ELEMS = 20;  
}
```

Interface – default methods

- Interface *default methods*
 - Methods tagged with the word `default`
 - Provide implementations by default, but which can be overridden in the classes that implement the *interface*

```
interface InterfaceB {  
    default int add(int i1, int i2) {  
        return i1 + i2;  
    }  
}
```

Interface – static methods

- Interface *static methods*
 - Methods tagged with the word `static`
 - Provide default implementations that cannot be overridden in classes that implement the *interface*

```
interface InterfaceC {  
    static void showPI() {  
        System.out.println(Math.PI);  
    }  
}
```

Superinterface and Subinterface

- An *interface* (subinterface) can derive from another *interface* (superinterface), extending the protocol
 - For this, the `extends` keyword must be used, as in inheritance among classes, but several interfaces can be extended

```
interface A { ... }  
  
interface B { ... }  
  
interface C extends A, B {  
    ...  
}
```

```
interface A { int n = 1; }  
  
interface B { int n = 2; }  
  
interface C extends A, B {  
    // int x = n + 4; // ERROR  
    int x = A.n + 4;  
}
```

```
interface A { int n = 1; }  
  
interface B { int p = 2; }  
  
interface C extends A, B {  
    int n = 5;  
    int x = A.n + p;  
}
```

Interface example

```
interface IntA {  
    void methodA(int nr);  
  
    static void methodB(int nr) {  
        System.out.println("Value + 10 = "+(nr+10));  
    }  
  
    default void methodC(int nr) {  
        System.out.println("Value - 10 = "+(nr-10));  
    }  
}
```

```
interface IntB extends IntA {  
    @Override  
    default void methodA(int nr) {  
        System.out.println("Value * 10 = "+(nr*10));  
    }  
}
```

```
interface IntC {  
    void methodA(float value);  
  
    static void methodB(int nr) {  
        System.out.println("Value + 5 = "+(nr+5));  
    }  
  
    default void methodC(int nr) {  
        System.out.println("Value - 5 = "+(nr-5));  
    }  
}
```

```
interface IntD extends IntB,IntC {  
    @Override  
    default void methodA(int nr) {  
        System.out.print("IntD::methodA... ");  
        IntB.super.methodA(nr);  
    }  
  
    @Override  
    default void methodC(int nr) {  
        System.out.println("IntD::methodC...");  
        System.out.print("\t");IntB.super.methodC(nr);  
        System.out.print("\t");IntC.super.methodC(nr);  
    }  
}
```

```
class ClassA implements IntD {  
    @Override  
    public void methodA(float value) {  
        System.out.println(  
            "ClassA:methodA(float): "+value  
        );  
    }  
}
```


Interface examples

- Example of use:

```
ClassA tst=new ClassA();  
tst.methodA(123);  
tst.methodA(1.23f);  
IntA.methodB(456);  
IntC.methodB(456);  
tst.methodC(789);
```

- Output:

```
IntD::methodA... Value * 10 = 1230  
ClassA:methodA(float): 1.23  
Value + 10 = 466  
Value + 5 = 461  
IntD::methodC...  
Value - 10 = 779  
Value - 5 = 784
```

Advanced Programming

Internal classes: *inner* and *nested*

Anonymous classes

Lambda expressions

Generic classes

Records

Internal classes: *inner* and *nested* classes

- In Java it is possible to define classes within classes
 - Defined in the same way as classes, but placed inside other classes or functions

```
class Exterior {  
    // ...  
    class Internal {  
        // ...  
    }  
}
```

- Outside the enclosing class, the internal class is referred to by its full name:

Exterior.Internal

Internal classes: *inner* and *nested* classes

- Internal classes may have appropriate access modes (public, protected, private or *package-private*) and/or be set to static

```
public class Exterior {  
    public Exterior() {}  
  
    public class Internal1 { }  
  
    public static class Internal2 { }  
}
```

Internal classes – *inner classes*

- The *inner classes* can only be accessed from an instance of the class where they are inserted in
 - The instances of these classes will be associated with the instance of the class where they are included
- They have access to the members of the class they belong to through the reference to that class:
`<class_exterior_name>.this`
- Externally, instances of these classes cannot be directly created, even if they are public
 - There may be an instance method of the *outer class* that returns instances of the *inner class*

Internal classes – *inner classes*

```
public class Exterior {
    int value;

    public Exterior(int value) { this.value = value; }

    @Override
    public String toString() { return "Exterior{" + value + '}'; }

    public Internal1 getObjectInternal1(int value) { return new Internal1(value); }

    // Inner class
    public class Internal1 {
        int value;

        public Internal1(int value) { this.value = value; }

        @Override
        public String toString() {
            return String.format("Internal1{%d, %d}", value, Exterior.this.value);
        }
    }
}
```

Internal classes – *nested classes*

- The *nested classes* are defined with the tag `static`
`[public] static class <nested_class> { ... }`
- Instances of these classes can be created directly, without the need for an instance of the *outer class* to support it

Internal classes – *nested classes*

```
public class Exterior {
    int value;

    public Exterior(int value) { this.value = value; }

    @Override
    public String toString() { return "Exterior{" + value + '}'; }

    // Nested class
    public static class Internal2 {
        int value;

        public Internal2(int value) {
            this.value = value;
        }

        @Override
        public String toString() {
            return "Internal2{" + value + '}'; // no direct access to Exterior.value
        }
    }
}
```


Internal classes – creating instances

- Example of using the classes from the previous examples:

```
public class Main {  
    public static void main(String[] args) {  
        Exterior a1 = new Exterior ( 100 );  
  
        //Exterior.Internal1 a2 = new Exterior.Internal1(200); /*ERROR!*/  
        Exterior.Internal1 a2 = a1.getObjectInternal1(200);  
  
        Exterior.Internal2 a3 = new Exterior.Internal2(300);  
  
        System.out.println(a1);  
        System.out.println(a2);  
        System.out.println(a3);  
    }  
}
```

Anonymous classes

- When the implementation of an *interface* or *abstract class* for a single use is wanted, corresponding to the creation of a single instance of an object, an *anonymous* implementation of the class can be created
 - This is done by associating *inline code* for each *abstract method* when creating the instance
 - Internally, a class with a unique name is created, for example, TestProtocol\$1

```
interface Protocol {
    void sendMessage(String msg);
    String receiveMessage();
}

class TestProtocol {
    public static void main(String[] args) {
        Protocol protocol = new Protocol() {
            @Override
            public void sendMessage(String msg) {
                System.out.printf("Sending... [%s]\n", msg);
            }

            @Override
            public String receiveMessage() {
                return "Message received.";
            }
        }; // Attention => ';'

        protocol.sendMessage("Test");
        System.out.println(protocol.receiveMessage());
    }
}
```

Anonymous classes

```
interface IShowInfo {
    void show(String info);
}

class ProcInfo {
    void run(IShowInfo showProgress, Object... objects) {
        for(var obj : objects) {
            // do something with obj
            if (showProgress!=null)
                showProgress.show(obj.toString());
        }
    }
}

class TestShowInfo {
    public static void main(String[] args) {
        ProcInfo procInfo = new ProcInfo();
        procInfo.run(
            new IShowInfo() {
                @Override
                public void show(String info) {
                    System.out.printf("Object processed: %s\n", info);
                }
            },
            123, 45.67, "DEIS-ISEC", true, List.of(10,9,8));
    }
}
```

Output :

```
Object processed: 123
Object processed: 45.67
Object processed: DEIS-ISEC
Object processed: true
Object processed: [10, 9, 8]
```

Lambda expressions

- When the interfaces to be implemented have only one *abstract method* (as in the previous example), a special case arises, which can be implemented using a *lambda expression*
- A *lambda expression* is an anonymous function (i.e., a function without a name) that can include a list of parameters and the code to be executed (note: it can also return a value)

([params]) -> statement(s)

Lambda expressions

- Example of *lambda expressions*:
 - `() -> System.out.println("DEIS-ISEC")`
 - `(int value) -> { return value > 0 && value <=10; }`
 - `(value) -> (value > 0 && value <=10)`
 - `value -> System.out.println(value)`

Lambda expressions

- Interfaces intended to be implemented as *lambda expressions* can have the `@FunctionalInterface` annotation to verify and ensure compliance with the requirements
 - It generates an error if it does not meet the requirements

Lambda expressions

@FunctionalInterface

```
interface IShowInfo {
    void show(String info);
}

class ProcInfo {
    void run(IShowInfo showProgress, Object ... objects) {
        for(var obj : objects) {
            // do something with obj
            if (showProgress != null)
                showProgress.show(obj.toString());
        }
    }
}

class TestShowInfo2 {
    public static void main(String [] args) {
        ProcInfo procInfo = new ProcInfo();
        procInfo.run(info -> System.out.printf("Object processed  %s\n",info),
            123,45.67,"DEIS-ISEC",true,List.of(10,9,8));
        procInfo.run((String value) -> {
            System.out.println(value);
            System.out.println("-".repeat(80));
        }, 123,456,789);
        procInfo.run(System.out::println , "Advanced Programming", "DEIS-ISEC");
    }
}
```

Output :

```
object processed : 123
object processed : 45.67
object processed : DEIS-ISEC
object processed : true
object processed : [10, 9, 8]
123
-----...----
456
-----...----
789
-----...----
Advanced programming
DEIS-ISEC
```

Generic classes – example

```
class MyType<T> {  
    T value;  
  
    public MyType(T value) { this.value = value; }  
  
    @Override  
    public String toString() { return "MyType{value=" + value + '}'; }  
}
```

```
public class App {  
    public static void main(String[] args) {  
        MyType<Integer> vi = new MyType<>(45);  
        MyType<Double> vd = new MyType<>(4.5);  
        MyType<String> vs = new MyType<>("forty-five");  
        MyType m1 = vd;  
        MyType m2 = vs;  
        for(var mt : Arrays.asList(vi,vd,vs))  
            System.out.println("> "+mt);  
        System.out.println(vd.getClass() == vs.getClass());  
        System.out.println(m1.value.getClass() == m2.value.getClass());  
        System.out.println(m1.getClass().toString());  
        System.out.println(m2.getClass().toString());  
        System.out.println(m1.value.getClass().toString());  
        System.out.println(m2.value.getClass().toString());  
    }  
}
```

Output:

```
> MyType{value=45}  
> MyType{value=4.5}  
> MyType{value=forty-five}  
true  
false  
class pt.isec.pa.generics.MyType  
class pt.isec.pa.generics.MyType  
class java.lang.Double  
class java.lang.String
```


Lambda expressions and generic types

```
@FunctionalInterface
interface IProcObject <T> {
    T proc (T value);
}

@FunctionalInterface
interface ICheckEnd <T> {
    boolean check(T value);
}

@FunctionalInterface
interface IUpdateStep <T> {
    void update(int step, T object);
}

class ProcSeries <T> {
    T object ;

    public ProcSeries(T object) {
        this.object = object ;
    }

    void proc(IProcObject <T> procObject,
             ICheckEnd <T> verifyEnd,
             IUpdateStep <T> updateStep) {
        int i=0;
        while (!verifyEnd.check(object)) {
            object = procObject.proc(object);
            updateStep.update(++i,object);
        }
    }
}
```

```
class TestProcSeries {
    public static void main(String[] args) {
        new ProcSeries<String>("PA_")
            .proc(
                obj -> obj.repeat(2),
                obj -> obj.length() > 30,
                (progress, object) -> {
                    System.out.printf("%3d: %s\n", progress, object);
                }
            );
        new ProcSeries<Integer>(0)
            .proc(
                value -> -(value > 0 ? value + 1 : value - 1),
                v -> v != 0 && v % 100 == 0,
                (progress, object) -> {
                    System.out.printf("%3d: %d\n", progress, object);
                }
            );
    }
}
```

Output:

```
1: PA_PA_
2: PA_PA_PA_PA_
3: PA_PA_PA_PA_PA_PA_PA_PA_
4:
PA_PA_PA_PA_PA_PA_PA_PA_PA_PA_PA_PA_PA_PA_PA_PA_
1:1
2: -2
3:3
...
99: 99
100: -100
```

records

- A `record` class provides a convenient way to define a data-holding class
 - The data within a `record` class is immutable and defined via the constructor
 - Automatic *getters* are generated for field access
 - Proper implementations of `equals()` and `hashCode()` methods are provided, taking into consideration all fields
 - A default `toString()` method is provided, which returns a string representation of all fields
- `records` are defined using the syntax:

```
record <record_name>(<parameter_list>) {}
```

records

- **Example:**

```
record Person(String name, int age) {}
```

in other class:

```
Person a = new Person("Curtis", 68);  
System.out.println(a.name());  
System.out.println(a.age());
```

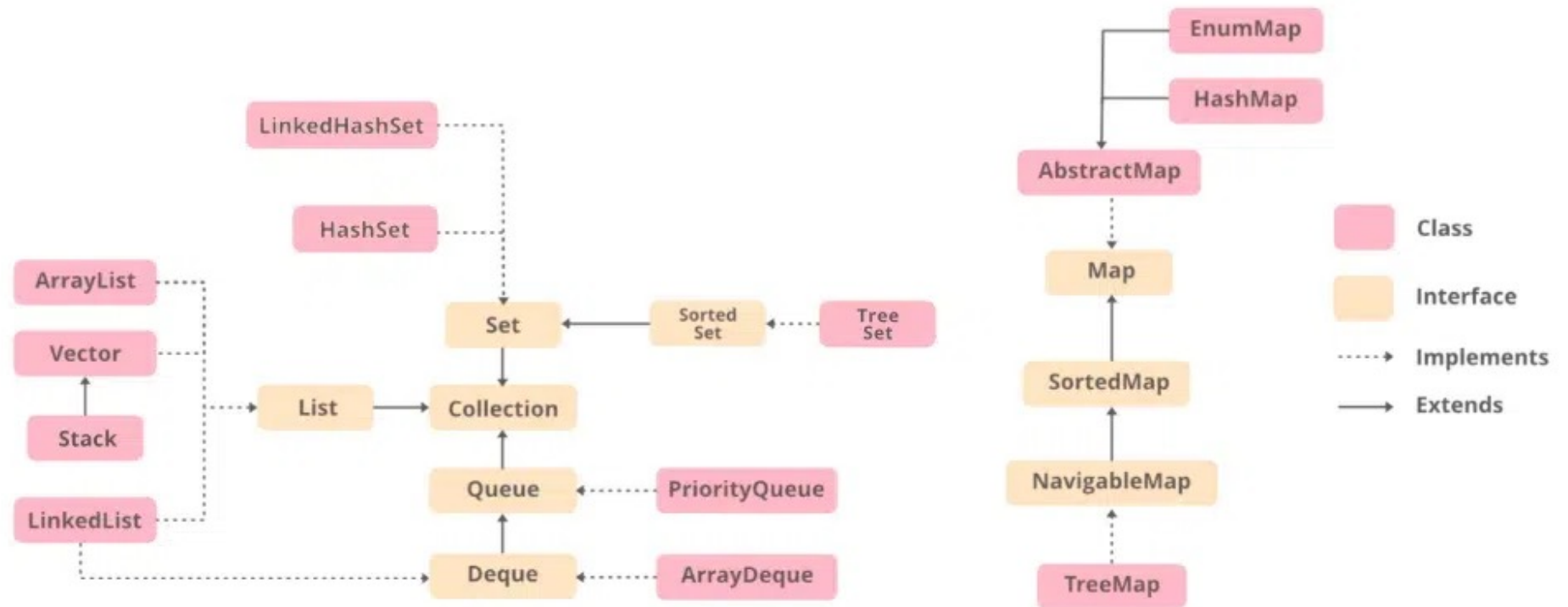
Advanced Programming

Collections

Collections

- The Java language provides a comprehensive set of classes designed to facilitate data collection processing by accommodating various paradigms:
 - *Lists*: for sequential access and manipulation of elements
 - *Sets*: for managing unique elements without duplicates
 - *Maps*: for storing key-value pairs, facilitating efficient data retrieval and manipulation

Collections



List

- The `List` interface defines essential functionalities for managing a dynamic list of objects by providing a set of appropriate functions, such as:
 - `get`, `size`
 - `add`, `addAll`
 - `remove`, `removeAll`, `removeIf`, `clear`
 - `indexOf`, `contains`, `lastIndexOf`
 - `iterator`, `forEach`
 - `sort`
 - `toArray`, `toString`
 - ...

List implementations

- `ArrayList`
 - Most commonly used implementation
 - Provides dynamic array functionality
- `LinkedList`
 - Doubly-linked list implementation
 - Suitable for frequent insertions and deletions
- `Vector`
 - Synchronized implementation
 - Suitable for multi-threaded environments
- `Stack`
 - Represents a *Last-In, First-Out* (LIFO) stack
 - Extends `Vector`

ArrayList<T>

- When creating an `ArrayList` instance, it must be specified the most generic type of objects it will contain
 - The `ArrayList` class is defined using the concept of 'generic classes'
- Example:

```
ArrayList<String> list1 = new ArrayList<>();  
list1.add("DEIS-ISEC");
```

```
List<Object> list2 = new ArrayList<>();  
list2.add("Advanced Programming");  
list2.add(1234);
```

```
for( int i = 0 ; i < list2.size(); i++){  
    // ... list2.get(i)  
}
```

```
for(String s: list1){  
    // ... s;  
}
```

Set<T>

- A `Set` allows for the storage and management of a series of non-repeated elements
 - Two items, *i1* and *i2*, are considered repeated when comparing the two using the `equals` method, `i1.equals(i2)`, returns `true`
 - It is essential that the `hashCode` methods are properly implemented so that operations on a `Set` lead to the expected results
 - If the indexing of the elements through the *hash code* is not done correctly, the search for equal elements will not work, and consequently, the mechanism that ensures the absence of repeated elements will also not work

Set<T>

- The `Set` interface defines the protocol associated with this type of collection, including methods such as:
 - `add`, `addAll`, `clear`, `contains`, `containsAll`, `isEmpty`, `iterator`, `remove`, `removeAll`, `size`, `toArray`, ...
- The preservation of the insertion order of elements in a `Set` is not always guaranteed
- Access to elements is performed through an iterative process over the collection elements

Set implementations

- HashSet
 - Most commonly used
 - Does not maintain insertion order
- EnumSet
 - Specifically designed for *enum* types
 - Iterates over elements in their natural order
- TreeSet
 - Maintains elements in sorted order
 - Suitable for sorted sets or range-based operations
- LinkedHashSet
 - Preserves insertion order
 - Provides predictable iteration order

HashSet and hashCode()

```
class Item {
    private static int count = 0;
    int i;

    public Item(int i) { this.i = i; }

    @Override
    public int hashCode() { return i; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null) return false;
        if (getClass() != obj.getClass()) return false;
        Item other = (Item) obj;
        return (i == other.i);
    }
}

public class App {
    public static void main(String[] args) {
        Set<Item> set = new HashSet<>();
        System.out.println(set.add(new Item(1))); // true
        System.out.println(set.add(new Item(2))); // true
        System.out.println(set.add(new Item(1)); // false);
        System.out.println(set.size(); // 2);
    }
}
```

```
class Item {
    private static int count = 0;
    int i;

    public Item(int i) { this.i = i; }

    @Override
    public int hashCode() { return count++; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null) return false;
        if (getClass() != obj.getClass()) return false;
        Item other = (Item) obj;
        return (i == other.i);
    }
}

public class App {
    public static void main(String[] args) {
        Set<Item> set = new HashSet<>();
        System.out.println(set.add(new Item(1))); // true
        System.out.println(set.add(new Item(2))); // true
        System.out.println(set.add(new Item(1)); // true);
        System.out.println(set.size(); // 3);
    }
}
```

Map<K,V>

- A `Map` allows for the storage and management of *key-value* pairs
 - Keys, `K`, and values, `V`, can be of any non-primitive type
 - Duplicate keys are not allowed, but values can be repeated
- The `Map<K, V>` interface defines the protocol associated with this type of collection
 - `clear`, `containsKey`, `containsValue`, `get`, `getOrDefault`, `isEmpty`, `keySet`, `put`, `putAll`, `putIfAbsent`, `remove`, `replace`, `size`, `values`, ...
- If a new value is specified for an existing key, via `put` or `replace`, the old value will be replaced by the new one
 - The previous value is returned in both indicated functions

Map implementations

- `HashMap`
 - Most commonly used
 - Does not maintain insertion order
- `EnumMap`
 - Specifically designed for *enum* keys
 - Iterates over entries in the order corresponding to the *enum* constants' natural order
- `TreeMap`
 - Maintains entries in sorted order
 - Suitable for sorted maps or range-based operations
- `LinkedHashMap`
 - Preserves insertion order
 - Provides predictable iteration order
- `Hashtable`
 - Thread-safe implementation, providing synchronization on all methods
 - Suitable for concurrent use by multiple threads

Map example

```
Map<String,Integer> colors = new HashMap<>();
```

```
colors.put("Orange",0xFF8C00);    // 0xFF8C00==16747520
colors.put("Yellow",0xFFFF00);    // 0xFFFF00==16776960
System.out.println(colors);
// Output: {Yellow=16776960, Orange=16747520}
```

```
colors.put("Orange",0xFF6500);    // 0xFF6500==16737536
System.out.println(colors);
// Output: {Yellow=16776960, Orange=16737536}
```

```
System.out.println(colors.keySet());
// Output: [Yellow, Orange]
System.out.println(colors.values());
// Output: [16776960, 16737536]
```

RGB Colours	
0xFF8C00 0	
0xFFFF00 0	
0xFF6500 0	

Advanced Programming

Enumerations

enum

- As in C or C++, the `enum` allows the definition of a set of constants that improve code readability
- By convention, `enums` like constants are written in uppercase

```
enum Planet {  
    MERCURY, VENUS, EARTH, MARS,  
    JUPITER, SATURN, URANUS, NEPTUNE  
}  
...  
Planet p = Planet.EARTH;  
...  
if (p != Planet.URANUS) ...  
...
```

enum

- As in Java everything is an object, including *enums*, which are derived from the abstract class Enum, *enums* also allow the use of members that provide additional information about them

```
enum Planet { MERCURY, VENUS, EARTH, MARS, JUPITER, SATURN, URANUS, NEPTUNE }
```

```
class TestPlanets {  
    public static void main(String [] args) {  
        System.out.println(Arrays.toString(Planet.values()));  
        for(var planet : Planet.values()) {  
            System.out.printf("%-10s [%d]", planet, planet.ordinal());  
            switch (planet) {  
                case EARTH -> System.out.println(" Beautiful");  
                case JUPITER -> System.out.println(" Big");  
                case MERCURY, VENUS -> System.out.println(" Hot");  
                default -> System.out.println();  
            }  
        }  
    }  
}
```

Output:

```
[MERCURY, VENUS, EARTH, MARS, JUPITER, SATURN, URANUS, NEPTUNE]  
MERCURY [0] Hot  
VENUS [1] Hot  
EARTH [2] Beautiful  
MARS [3]  
JUPITER [4] Big  
SATURN [5]  
URANUS [6]  
NEPTUNE [7]
```

Custom enum

- Enumerations can have
 - variable members
 - methods that may or may not be abstract
 - Enumerations can implement interface

```
enum Planet {  
    MERCURY(2440), VENUS(6052), EARTH(6371), MARS(3390),  
    JUPITER(69911), SATURN(58232), URANUS(25362), NEPTUNE(24622);  
  
    private int radius; // km  
  
    Planet(int radius) { this.radius = radius; }  
  
    public int getRadius() { return radius; }  
  
    @Override  
    public String toString() {  
        String str = super.toString().toLowerCase();  
        str = Character.toUpperCase(str.charAt(0)) + str.substring(1);  
        return str ;  
    }  
}
```

Custom enum

- For the previous example enum Planet...

```
class TestPlanets {  
    public static void main(String [] args) {  
        System.out.println(Arrays.toString(Planet.values()));  
        for(var planet : Planet.values())  
            System.out.printf("%-10s [%d] (%5d km)\n",  
                               planet, planet.ordinal(), planet.getRadius());  
    }  
}
```

Output:

```
[Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, Neptune]  
Mercury [0] ( 2440 km)  
Venus   [1] ( 6052 km)  
Earth   [2] ( 6371 km)  
Mars    [3] ( 3390 km)  
Jupiter [4] (69911 km)  
Saturn  [5] (58232 km)  
Uranus  [6] (25362 km)  
Neptune [7] (24622 km)
```

enum – abstract methods

```
enum Operation {  
    add { double calculate(double x, double y) { return x + y; } },  
    sub { double calculate(double x, double y) { return x - y; } },  
    mul { double calculate(double x, double y) { return x * y; } },  
    div { double calculate(double x, double y) { return x / y; } };  
  
    abstract double calculate(double x, double y);  
}  
  
class TestOperation {  
    static void f(double x, double y) {  
        for(Operation op : Operation.values()) {  
            System.out.println(x + " " + op + " " + y + " = " + op.calculate(x, y));  
        }  
    }  
  
    public static void main(String [] args) {  
        f(12.0, 4.0);  
    }  
}
```

Output:

```
12.0 add 4.0 = 16.0  
12.0 sub 4.0 = 8.0  
12.0 mul 4.0 = 48.0  
12.0 div 4.0 = 3.0
```

enum – method redefinition

```
enum Operation {
    add {
        double calculate(double x, double y) { return x + y; }

        @Override
        public String toString() { return "plus"; }
    },
    sub { double calculate(double x, double y) { return x - y; } },
    mul { double calculate(double x, double y) { return x * y; } },
    div { double calculate(double x, double y) { return x / y; } };

    abstract double calculate(double x, double y);
}

class TestOperation {
    static void f(double x, double y) {
        for(Operation op : Operation.values()) {
            System.out.println(x + " " + op + " " + y + " = " + op.calculate(x, y));
        }
    }

    public static void main(String [] args) {
        f(8.0,5.0);
    }
}
```

Output:

```
8.0 plus 5.0 = 13.0
8.0 sub 5.0 = 3.0
8.0 mul 5.0 = 40.0
8.0 div 5.0 = 1.6
```

Advanced Programming

Exceptions

Exceptions

- An exception is an abnormal condition (an error) that occurs during the execution of a program, which can be handled in the code
- When an exception happens...
 - an object is created that represents this exception
 - the originating instruction is interrupted
 - the object is sent to the code in question, for eventual processing

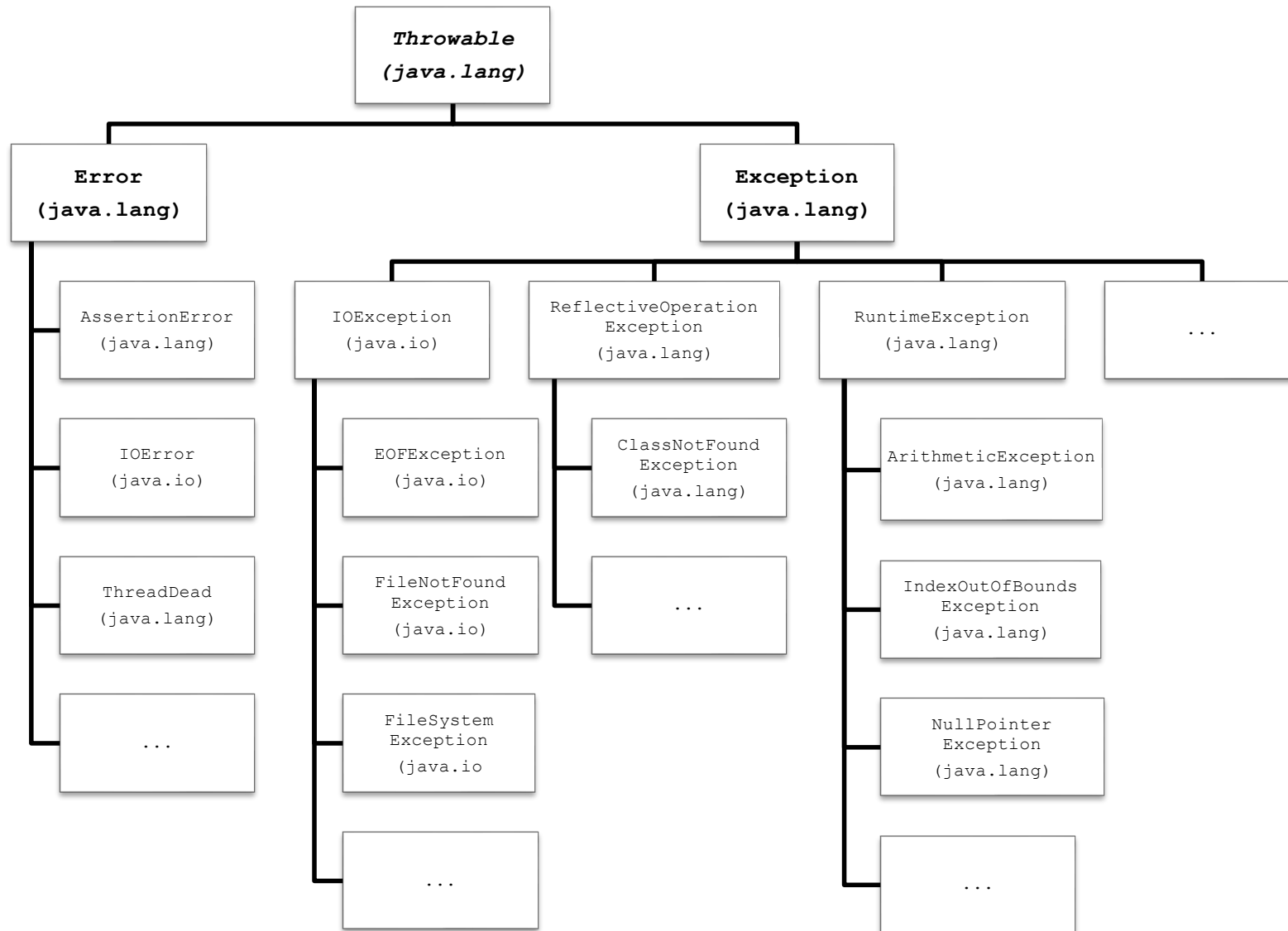
Exceptions

- The code can either handle the exception or redirect/return it for handling by the calling function
 - This process is continued until it reaches the base of the program (`main` function)
 - If not properly treated, it ends up causing the interruption of the program

Throwable class

- All objects representing situations that can interrupt a program have as their base class: `Throwable`
- `Throwable` class has two derived classes:
 - `Error` – severe errors that should not be handled
 - `Exception` – errors that an application can (should) handle
- Both classes serve as the basis for many specializations (more concrete representation of various anomalous situations)

Errors and exceptions



Unhandled Exceptions

- As already mentioned, when an exception is not handled, it causes the program to be interrupted
 - After interrupting the program, the JVM displays the available information about the exception
 - It includes information about the function calling sequence and respective lines of code

```
1 package pt.isec.pa.exceptions;
2
3 public class Main {
4     public static void main(String [] args) {
5         exceptions ();
6     }
7
8     static void exceptions() {
9         int i = 1234/(11-22/2);
10        System.out.println("i="+i);
11    }
12 }
```

Output:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at pt.isec.pa.exceptions.Main.exceptions (Main.java:9)
    at pt.isec.pa.exceptions.Main.main (Main.java:5)
```

Exception handling

- Exception handling is performed using *try-catch* blocks:

```
try { ... Instructions that may throw exceptions ... }  
catch (<Exception> <variable>) { ... } // 1 or more  
finally { ... } // optional
```

- Statements that might throw exceptions are placed inside the `try` block
- Exceptions are handled in `catch` blocks, and multiple `catch` blocks can be used to handle different types of exceptions
- The `finally` block is optional. If defined, the `finally` block executes whether an exception occurs or not
 - Even if there is a return in the previous blocks, the `finally` block is always executed

Exception handling

```
public class Main {
    public static void main(String [] args) {
        exceptions();
    }

    static void exceptions() {
        System.out.println("begin");
        try {
            int i = 1234/(11-22/2);
            System.out.println("i="+i);
        } catch ( ArithmeticException e) {
            System.out.println("Error: "+e);
            return;
        } finally {
            System.out.println("Free resources and continue");
        }
        System.out.println("end");
    }
}
```

Output:

```
begin
Error: java.lang.ArithmeticException : / by zero
free resources and continue
```

Exception handling

- When the code performed can trigger several exceptions:
 - Place several *catch blocks*, putting the most specific ones first and the most generic ones last
 - In the presence of an exception, the *catch* blocks are processed in the order in which they were defined to find a *catch* with a parameter suitable for the exception that occurred
 - `exception instanceof <CatchParameterClass>`
 - If the most generic one is put first, the least generic one(s) will never be used (IDEs normally detect and warn about this situation)

Exception handling

- When the same processing applies to multiple exceptions, possible approaches include:
 - Using a more generic `Exception` class
 - Aggregating exceptions in a single catch block:

```
try {  
    ...  
} catch (<ExceptionA> | <ExceptionB> | ... e) {  
    ...  
}
```

Exception handling

```
// ...
static void exceptions() {
    int debug=0,i;
    try {
        debug = 1;
        System.out.print("Number: ");
        int v = new Scanner(System.in).nextInt();
        debug += 20;
        i = 1234/v;
        debug += 300;
    } catch (ArithmeticException e) {
        System.out.println("Error1: "+e);
        i = Integer.MAX_VALUE ;
    } catch (InputMismatchException e ) {
        System.out.println("Error2: "+e);
        return;
    } catch (Exception e) {
        System.out.println("Error3: "+e);
        return;
    } finally {
        System.out.println("debug = " + debug);
    }
    System.out.println("i="+i);
}
// ...
```

Examples of execution result:

```
Number: 123
debug = 321
i=10
-----
Number: 0
Error1: java.lang.ArithmeticException:/by
zero
debug = 21
i=2147483647
-----
Number: DEIS-ISEC
Error2: java.util.InputMismatchException
debug = 1
-----
Number: ^D
Error3: java.util.NoSuchElementException
debug = 1
```

Exception handling

```
// ...
static void exceptions() {
    int debug=0,i;
    try {
        debug = 1;
        System.out.print("Number: ");
        int v = new Scanner(System.in).nextInt();
        debug += 20;
        i = 1234/v;
        debug += 300;
    } catch (ArithmeticException | InputMismatchException e) {
        System.out.println("Error (" + e + "), assumes 0");
        i = 0;
    } catch (Exception e) {
        System.out.println("Error: " + e);
        return;
    } finally {
        System.out.println("debug = " + debug);
    }
    System.out.println("i=" + i);
}
// ...
```

throw

- An application can, at any time, throw an exception of a certain type using the `throw` command. Examples:
 - `throw new Exception();`
 - `throw new Exception("what happened");`
- After executing this command, the code is immediately interrupted and the handling of exceptions that may exist is started or the program is interrupted
- An application can forward a handled exception to another handler performed elsewhere in the program

```
...  
} catch (Exception e) {  
    // something  
    throw e;  
}
```

throw

```
static void exceptions() {  
    int i;  
    try {  
        if (System.nanoTime() > 1000) throw new Exception("Test");  
        i = 1234/0;  
    } catch (ArithmeticException e) {  
        i = 0;  
    } catch (Exception e) {  
        System.out.println("Error: " + e.getMessage());  
        return;  
    }  
    System.out.println("i="+i);  
}
```

Output:

Error: Test

throws

- When a function does not process a certain exception, it must be explicit about this behavior by including the `throws` clause in the function declaration
 - More than one exception can be indicated (list separated by ' , ')
- The code that calls the function is responsible for handling it

```
static void exceptions() throws ArithmeticException {  
    int i = 1234 / 0;  
    System.out.println("i="+i);  
}
```

Custom exception

- In the context of a program it is possible to create new types of exception
 - Derive the new class from the base `Exception` class or one of its more convenient subclasses

```
class MyException extends Exception {  
    public MyException() { }  
    public MyException(String message) {  
        super(message);  
    }  
}
```

```
static void exceptions() {  
    try {  
        if (System.nanoTime() % 2 == 0)  
            throw new MyException();  
        else  
            throw new MyException("Second version");  
    } catch (MyException me) {  
        me.printStackTrace();  
    }  
}
```

Output1

```
pt.isec.pa.exceptions.MyException  
    at pt.isec.pa.exceptions.Main.exceptions  
(Main.java:28)  
    at pt.isec.pa.exceptions.Main.main (Main.java:11)
```

Output2

```
pt.isec.pa.exceptions.MyException : Second version  
    at pt.isec.pa.exceptions.Main.exceptions (Main.java:30)  
    at pt.isec.pa.exceptions.Main.main (Main.java:11)
```

Advanced Programming

Files

Files

- In *Java*, a file or directory in the file system is represented by a `File` object
 - Example: `File file = new File("pa_info.txt");`
- Through the methods provided by the `File` object, it is possible to perform the most common tasks on files and directories. Examples:
 - `canExecute`, `canRead`, `canWrite`
 - `exists`, `isFile`, `isHidden`, `isDirectory`
 - `length`, `getPath`, `getAbsolutePath`
 - `createNewFile`, `createTempFile`
 - `delete`, `renameTo`
 - `mkdir`, `mkdirs`, `list`, `listFiles`

Files and Exceptions

- Most file operations are subject to many exceptions, due to their nature
 - A *Java* program that uses files must handle all exceptions that might be triggered by the respective operations
- Most common exceptions:
 - `IOException`
 - `EOFException`
 - `FileNotFoundException`
 - `FileSystemException`
 - `InterruptedException`
 - `ObjectStreamException`
 - `SocketException`
 - `UnsupportedDataFormatException`

Files: reading and writing

- In *Java*, reading and writing files rely on the concept of *Streams*
- A *stream* represents a sequence of data flowing between a source and a destination
 - In file reading, the source is the file, and the destination is a variable or memory block (e.g., a byte array)
 - In file writing, the source is a variable or memory block, and the destination is the file
- This model extends beyond files, supporting shared memory access, socket communication, and other data exchanges

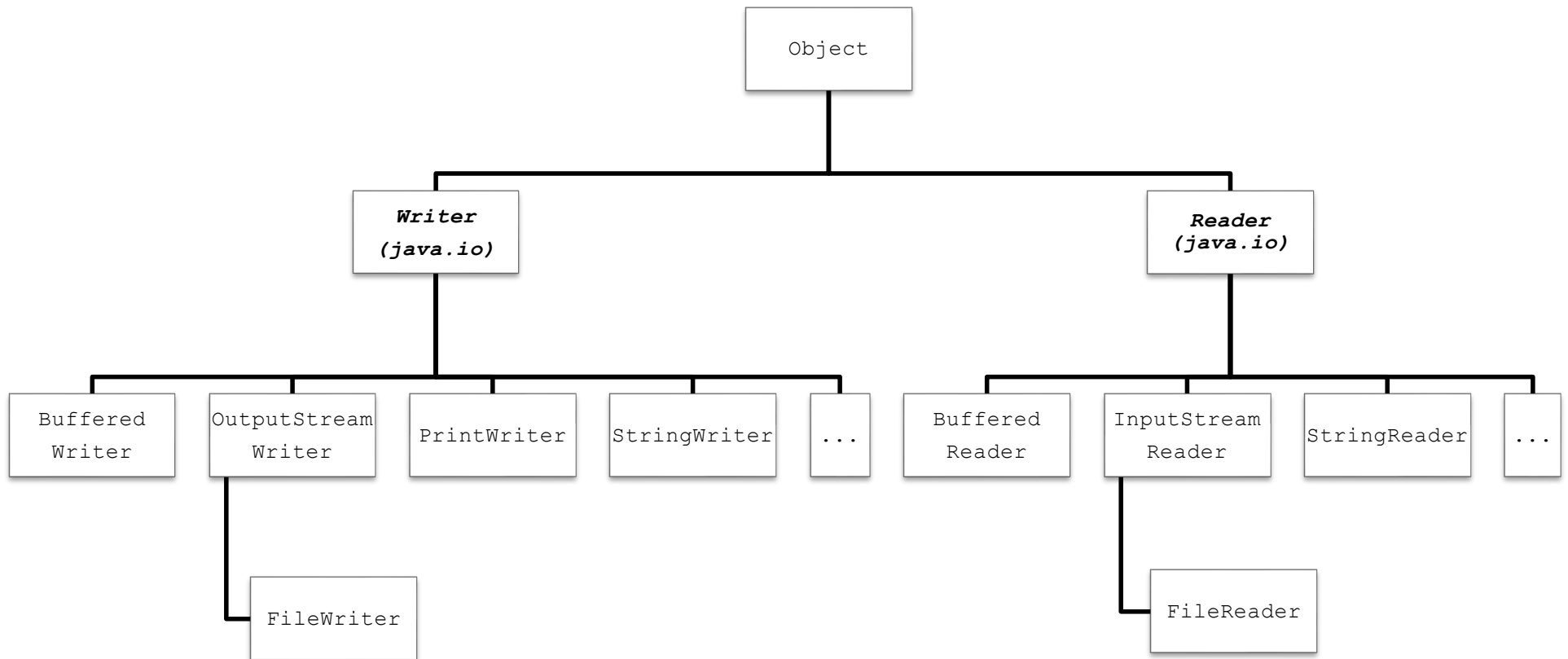
Files: reading and writing

- Functionalities for reading and writing operations are provided by different classes, structured in layers
- Depending on the application's needs (e.g., reading byte blocks, interpreting text lines, or buffering operations), these classes complement each other to offer the required functionalities

Files: reading and writing

- In Java, a clear distinction is made between *streams* that work with *characters* (text) or *bytes* (binary information) and whether they are for reading or writing
 - *Character streams* (text)
 - Reader
 - Writer
 - *Byte streams* (binary)
 - InputStream
 - OutputStream

Character *streams*



Text Files – Writing

- File opening for writing with `FileWriter`...

```
File f = new File("deis.txt");  
FileWriter fw = new FileWriter(f);
```

or

```
FileWriter fw = new FileWriter("deis.txt");
```

- Most common `FileWriter` methods:
 - `write(<String> | <char []> | <int>);`
 - `flush();`
 - `close();`

Text Files – Writing

- To obtain more features some utility classes can be used, for example:

- `BufferedWriter`

```
BufferedWriter bw = new BufferedWriter(<Writer>);
```

- Adds *buffering* functionalities to `Writer`
- Provides the methods: `write`, `flush`, `newline`, `close`

Text Files – Writing

- To obtain more features some utility classes can be used, for example (cont.):

- `PrintWriter`

```
PrintWriter pw = new PrintWriter(<Writer>|<File>|<String>);
```

- The version that takes a `String` (file name) automatically creates an `OutputStreamWriter/ FileWriter`
- Provides the methods: `print`, `println`, `printf`, `format`, `write`, `flush`, `close`

Text Files – Writing

```
try {  
    File f = new File("deis.txt");  
    FileWriter fw = new FileWriter(f);  
    BufferedWriter bw = new BufferedWriter(fw);  
    PrintWriter pw = new PrintWriter(bw);  
  
    pw.println("DEIS-ISEC");  
  
    pw.close();  
  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

Text Files – Writing

```
PrintWriter pw = null;
try {
    pw = new PrintWriter(
        new BufferedWriter(
            new FileWriter("deis.txt")
        )
    );

    pw.println("DEIS-ISEC");

} catch (IOException e) {
    e.printStackTrace();
} finally {
    if (pw != null)
        pw.close();
}
```

Text Files – Reading

- Opening file for reading with `FileReader`...

```
File f = new File("deis.txt");  
FileReader fr = new FileReader(f);
```

or

```
FileReader fr = new FileReader("deis.txt");
```

- `FileReader`'s most common methods:
 - `int read();`
 - `read(<CharBuffer> | <char []>);`
 - `close();`

Text Files – Reading

- To get access to additional features, a `BufferedReader` can be used

```
BufferedReader br = new BufferedReader (<Reader>) ;
```

- Adds *buffering* capabilities to `Reader`
- Provides the methods:
 - `readline`, `read`, `close`
 - `lines()` which returns a `Stream<String>`

Text Files – Reading

- To make reading easier, a `Scanner` object can be used:

```
FileReader fr = new FileReader("deis.txt");  
BufferedReader br = new BufferedReader(fr);  
Scanner scanner = new Scanner(br);
```

- `Scanner` methods are used for reading operations

Text Files – Reading

```
try {  
    FileReader fr = new FileReader("deis.txt");  
    BufferedReader br = new BufferedReader(fr);  
  
    String str = br.readLine();  
  
    br.close();  
  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

Text Files – Reading

```
public static String readFirstLine() throws IOException {  
    BufferedReader br = null;  
    try {  
        FileReader fr = new FileReader("deis.txt");  
        br = new BufferedReader(fr);  
  
        return br.readLine();  
    } catch (IOException e) {  
        return null;  
    } finally {  
        if (br != null)  
            br.close();  
    }  
}
```


Closing files

- All files must be closed after use
 - When a file is being used through *streams*, all that needs to be done is to close one of the objects involved in its management for the support *stream* to be closed
 - Note: the `Scanner` object, that possibly was used to access the file, must be closed autonomously

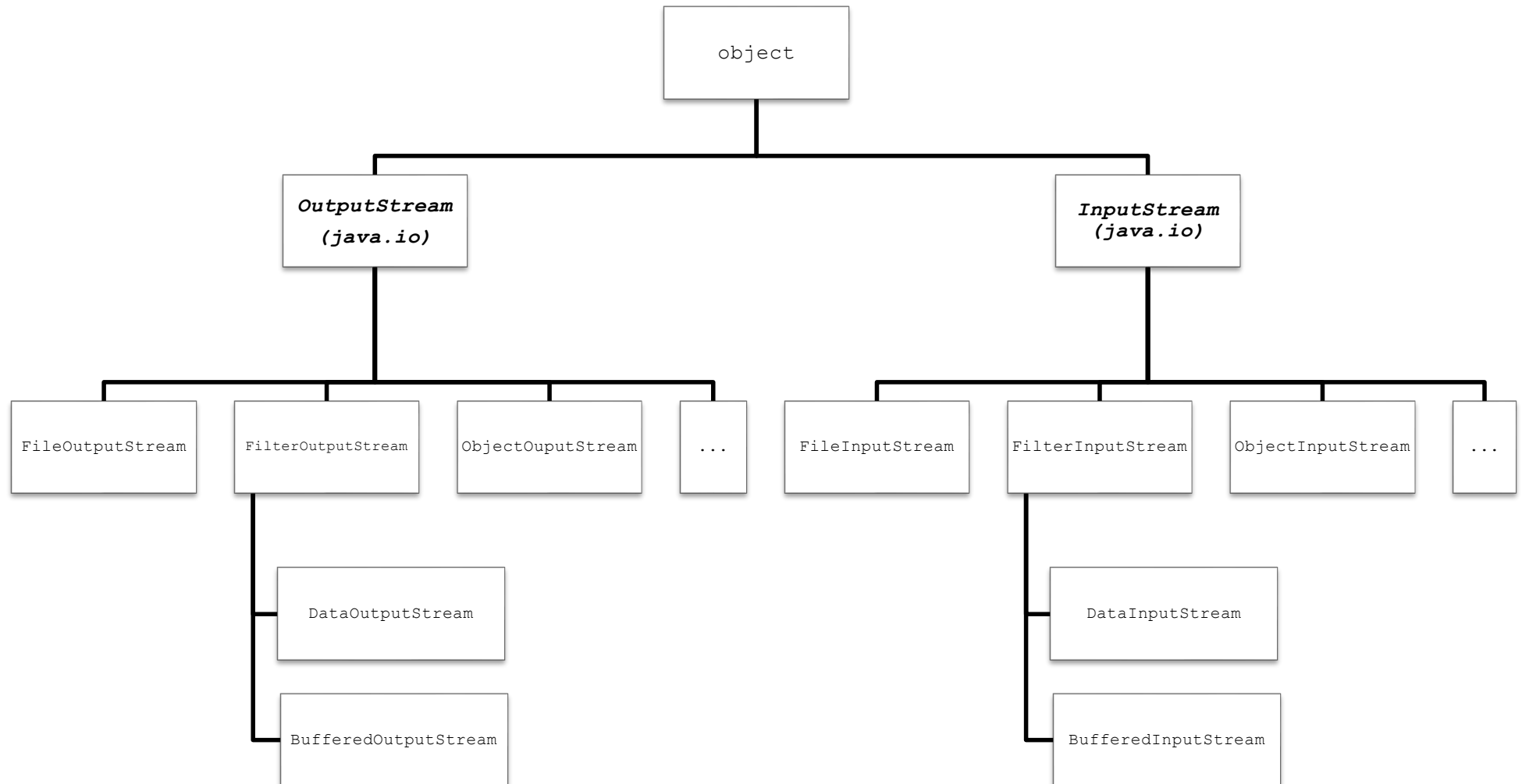
AutoCloseable and *try-with-resources*

- Objects that implement the `AutoCloseable` interface can be managed more easily using a *try-with-resources* command
 - The involved *streams* are automatically closed at the end of the *try-catch* block

```
public static String readFirstLine() {  
    try (FileReader fr = new FileReader(testFile);  
        BufferedReader br = new BufferedReader(fr)) {  
  
        return br.readLine();  
  
    } catch (IOException e) {  
        return null;  
    }  
}
```

- From the JDK 1.4, the `java.nio` package with additional classes was made available to facilitate some operations on files
 - `Path` – (*interface*) represents a path to a given file
 - `Paths.get(...)` – returns a `Path` object for the given resource. It receives a URI or a set of strings that, concatenated, constitute the file name
 - `Files` – utility class that provides several methods for working with files and directories, based on the `Path` that identifies them
 - `copy`, `createFile`, `createDirectory`, `createTempFile`, ...
 - `delete`, `deleteIfExists`, ...
 - `isDirectory`, `isRegularFile`, `isHidden`, `isExecutable`, ...
 - `lines`, `readAllLines`, `readAllBytes`, ...
 - `write`, `writeString`, ...

Byte streams



FileOutputStream

- This class allows the creation of basic objects that permit the writing operations to binary files
- `FileOutputStream`:
 - `write(int)` – write one *byte*
 - `write(byte[])` – write a set of *bytes*
 - `flush()` – flush the *buffers*
 - `close()` – close the file

FileInputStream

- This class allows the creation of basic objects that permit reading operations from binary files
 - `FileInputStream`:
 - `int read()` – read one *byte*
 - `int read(byte[])` – read a set of *bytes*
 - there is another version of the method that allows reading just a few bytes for an array
 - `int available()` – check how many *bytes* can be read before getting stuck on hold
 - `long skip(long)` – skip a set of *bytes*
 - `close()` – close the file

DataInputStream / DataOutputStream

- DataInputStream

- read
- readBoolean
- readByte
- readChar
- readDouble
- readFloat
- readInt
- readUTF
- readAllBytes
- available
- close
- ...

- DataOutputStream

- write
- writeBoolean
- writeByte
- writeBytes
- writeChar
- writeChars
- writeDouble
- writeFloat
- writeInt
- writeUTF
- writeAll
- flush
- ...

Byte Files – Write

```
int        i = 10;
float      f = 11.1f;
double     d = 22.22;
try ( DataOutputStream dos =
      new DataOutputStream(
        new BufferedOutputStream(
          new FileOutputStream("deis.bin"))) )
{
    dos.writeInt(i);
    dos.writeFloat(f);
    dos.writeDouble(d);
} catch (IOException e) {
    e.printStackTrace();
}
```


Byte Files – Read

```
int      temp_i = 0;
float    temp_f = 0f;
double   temp_d = 0;
try ( DataInputStream dis =
      new DataInputStream(
        new BufferedInputStream(
          new FileInputStream("deis.bin"))) )
{
    temp_i = dis.readInt();
    temp_f = dis.readFloat();
    temp_d = dis.readDouble();
} catch (IOException e) {
    e.printStackTrace();
}
```

ObjectInputStream/ObjectOutputStream

- These objects provide methods similar to `DataInputStream/DataOutputStream`, but additionally they also allow reading and writing objects through a technique called *Serialization/Deserialization*
 - `ObjectOutputStream.writeObject (Object o) ;`
 - *Serialization* – allows writing an object through a sequence of *bytes*, including information about the data type
 - `Object o = ObjectInputStream.readObject () ;`
 - *Deserialization* – allows reading a set of *bytes* that permits reconstructing a previously written object
 - No constructor is executed when an object is restored through deserialization

Serializable

- For a certain type of object to be the target of a serialization process, it has to implement the `Serializable` interface

```
class MyObject implements Serializable { ... }
```

- For a class to be serializable, all of its member variables must be of primitive types or are references to other objects that are also serializable
 - `Exception NotSeriazableException` will be generated if they are not serializable
- `static` variables are not written
- Commonly used arrays and collections (`List`, `Set`, `Map`) are serializable

serialVersionUID

- All the classes that define serializable objects must have the member variable `serialVersionUID` (*static, final* and of type `long`) to indicate the version of the object, which is saved with the rest of the information

```
class MyObject implements Serializable {  
    @Serial  
    static final long serialVersionUID = 100L;  
    ...  
}
```

serialVersionUID

- When trying to replace an object, the version (present in the *stream*) is checked to see if it matches the version existing in the code
 - If the versions don't match, an exception is generated:
`InvalidClassException`
- If the `serialVersionUID` is not defined, then the value is generated automatically at each significant change of the class (member variables, constructors, ...), making it impossible to read objects serialized with previous versions
- Usually, the value is incremented with each change of the data that constitute the class

Serialization – Example

```
void save(String filename, MyObject obj) {  
    try (ObjectOutputStream oos =  
        new ObjectOutputStream(  
            new FileOutputStream(filename)))  
    {  
        oos.writeObject(obj);  
    } catch (Exception e) {  
        System.err.println("Error saving data");  
    }  
}
```

Deserialization – Example

```
MyObject load(String filename) {  
    try (ObjectInputStream ois =  
        new ObjectInputStream(  
            new FileInputStream(filename)))  
    {  
        return (MyObject) ois.readObject();  
    } catch (Exception e) {  
        System.err.println("Error loading data");  
    }  
    return null;  
}
```

transient

- Sometimes certain data is not intended to be included in the serialization/deserialization process
- These variables must be marked with the `transient` tag

```
class MyObject implements Serializable {  
    ...  
    transient String temp_s;  
    ...  
}
```


transient

- When an object is restored, the `transient` members will get the default values defined for the data type or the default values defined for the `final` members
- The `readObject` method can be redefined to allow the initialization of transient **variables**

```
public class MyClass implements Serializable {  
    private transient int myVar;  
  
    private void readObject(ObjectInputStream in)  
        throws IOException, ClassNotFoundException {  
        in.defaultReadObject();  
        myVar = 123;  
    }  
}
```

writeUnshared/readUnshared/reset

- During serialization operations, a *cache* of recorded objects is kept
- When the repeated recording of an object is requested, for example in a hierarchy of objects that repeatedly refer to another object, only the reference to the already recorded object is made

writeUnshared/readUnshared/reset

- This way, when the object is restored, this situation is identified and all objects replaced refer to the same object, as before the serialization process
- If this behavior is not the expected, the `writeUnshared` and `readUnshared` functions should be used to write multiple copies of the same object
 - The referred *cache* can be deleted by calling the `reset` method

Random access to files

- Despite the generalization used with *streams*, the Java language also allows random access to files through the `RandomAccessFile` class in the *package* `java.io`
- The class `RandomAccessFile` ...
 - allows reading and writing files, without the need to close and reopen them
 - In the constructor, in addition to the file name, the intention of whether to read only ("`r`") or read and write ("`rw`") must be indicated
 - manages the current read/write position and allows its change through the `getFilePointer` and `seek` methods
 - provides `read*` and `write*` methods similar to the methods of the `DataInputStream` and `DataOutputStream` classes